



第4章 ARM伪指令及编程基础

本章主要内容

- 1、伪指令概述
- 2、通用伪指令
- 3、与ARM指令相关的宏指令
- 4、ARM工程
- 5、ARM程序框架
- 6、ARM汇编语言程序设计
- 7、C/C++语言和汇编语言的混合编程。

本章教学要求

1、了解

- ARM程序框架
- C/C++语言和汇编语言的混合编程

2、理解

- 伪指令的用途、ARM工程

3、掌握

- 与ARM指令相关的宏指令、通用伪指令
- ARM汇编语言程序、C与汇编之间的函数调用

目前汇编有ARM ASM汇编和GNU ASM汇编，本课程主要讲 ARM ASM 汇编。

4.1 伪指令概述

1、什么是伪指令

人们设计了一些专门用于指导汇编器进行汇编工作的指令，由于这些指令不形成机器码指令，它们只是在汇编器进行汇编工作的过程中起作用，所以被叫做伪指令。

2、伪指令具有的两个特征

- (1) 伪指令是一条指令；
- (2) 伪指令没有指令代码。

4.1 伪指令概述

3、伪指令的作用

- (1) 程序定位的作用;
- (2) 为非指令代码进行定义;
- (3) 为程序完整性做标注;
- (4) 有条件的引导程序段。

4.2 通用伪指令

在 ARM 汇编程序语言中，有如下几种伪指令：

- 符号定义 (Symbol Definition) 伪指令
- 数据定义 (Data Definition) 伪指令
- 汇编控制 (Assembly Control) 伪指令
- 其它 (Miscellaneous) 伪指令

4.2 通用伪指令

4.2.1 为变量定义或赋值的伪指令

符号的命名由编程者决定，但必须遵循以下约定：

- 符号区分大小写，同名的大、小写符号会被编译器认为是两个不同的符号；
- 符号在其作用范围内必须唯一；
- 自定义的符号不能与系统保留字相同；
- 符号不应与指令或伪指令同名。

4.2 通用伪指令

一、声明全局变量伪指令 GBLA、GBLL和GBLS

GBLA、GBLL 和 GBLS 伪指令用于定义一个ARM 程序中的全局变量，并将其初始化。

指令格式：GBLA(GBLL和GBLS) <variable>

variable 为变量名称；

- GBLA 定义一个全局数字变量，其默认初值为 0；
- GBLL 定义一个全局逻辑变量，其默认初值为 FALSE（假）；
- GBLS 定义一个全局字符串变量，其默认初值为 空；

4.2 通用伪指令

例:

GBLA Test1 ; 定义一个全局数字变量, 变量名为 Test1

GBLL Test2 ; 定义一个全局逻辑变量, 变量名为 Test2

GBLS Test3 ; 定义一个全局字符串变量, 变量名为 Test3

全局变量 的变量名在整个程序范围内必须具有 唯一性。

4.2 通用伪指令

二、声明局部变量伪指令LCLA、LCLL和LCLS

LCLA、LCLL和LCLS伪指令用于定义一个ARM程序中的局部变量，并将其初始化。

格式：LCLA(LCLL和LCLS) <variable>

variable 为变量名称；

- LCLA 定义一个局部数字变量，其默认初值为 0；
- LCLL 定义一个局部逻辑变量，其默认初值为 FALSE (假)；
- LCLS 定义一个局部字符串变量，其默认初值为 空。

4.2 通用伪指令

例：

LCLA Test4 ; 定义一个局部数字变量，变量名为 Test4

LCLL Test5 ; 定义一个局部逻辑变量，变量名为 Test5

LCLS Test6 ; 定义一个局部字符串变量，变量名为 Test6

局部变量 的变量名在变量作用范围内必须具有唯一性。

在默认情况下，局部变量只在定义该变量的程序段内有效。

4.2 通用伪指令

三、变量赋值伪指令 SETA、SETL和SETS

伪指令SETA、SETL和SETS 用于给一个已经定义的全局变量或局部变量进行赋值。 注：要顶格写

指令格式：变量名 SETA (SETL或SETS) 表达式

SETA伪指令用于给一个数字变量赋值；

SETL伪指令用于给一个逻辑变量赋值；

SETS伪指令用于给一个字符串变量赋值；

4.2 通用伪指令

例：

Test1 SETA 0xAA ;将Test1变量赋值为0xAA。

Test2 SETL {TRUE} ;将Test2 变量赋值为真;
;假是{FALSE}。

Test3 SETS "Testing" ;将Test3变量赋值为 "Testing" 。

4.2 通用伪指令

四、定义寄存器列表伪指令

指令 LDM/STM 需要使用一个比较长的寄存器列表，使用伪指令 RLIST 可对一个列表定义一个统一的名称。

格式：<name> RLIST <{list}>

例如：

LoReg RLIST {R0-R7} ; 定义寄存器列表{R0-R7}
; 的名称为LoReg

STMFD SP!, LoReg ; 堆栈操作使用寄存器列表

RegList RLIST {R0-R5,R8,R10} ; 将寄存器列表名称定义
; 为RegList, 可在ARM指令LDM/STM中
; 通过该名称访问寄存器列表

4.2 通用伪指令

4.2.2 数据定义伪指令

1、LTORG

用于声明一个数据缓冲池（文字池）的开始。

语法格式：LTORG

伪指令 LTORG 用来说明某个存储区域为一个用来暂存数据的数据缓冲区，也叫文字池或数据缓冲池。大的代码段也可以使用多个数据缓冲池。

4.2 通用伪指令

例:

AREA example, CODE, READONLY

Start BL Func1

...

Func1 LDR R1,=0x800

MOV PC,LR

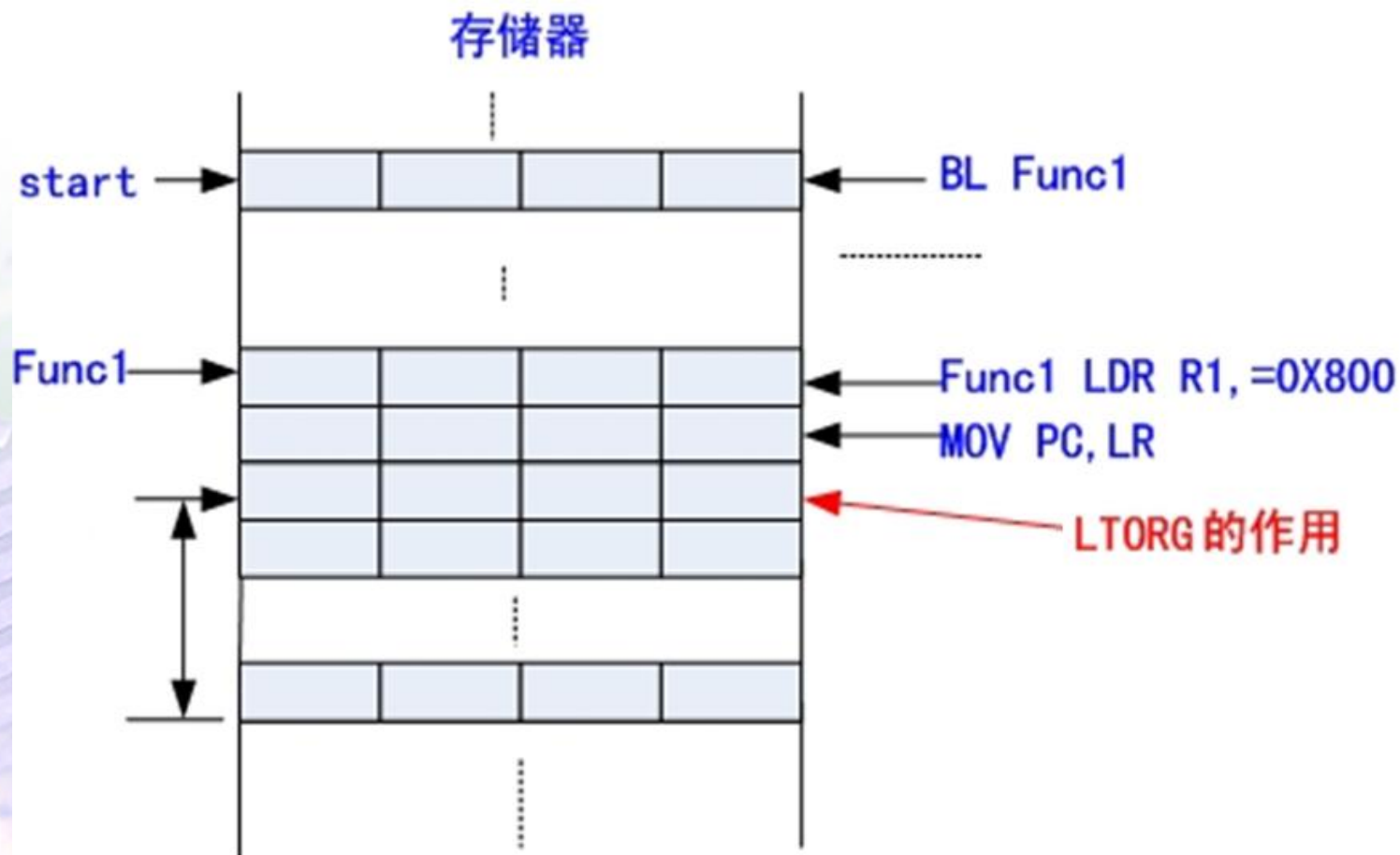
LTORG ; 定义数据缓冲池的开始位置,
; 系统会自动设置数据缓冲池的大小

...

END

4.2 通用伪指令

下图为 LTORG 的作用。



4.2 通用伪指令

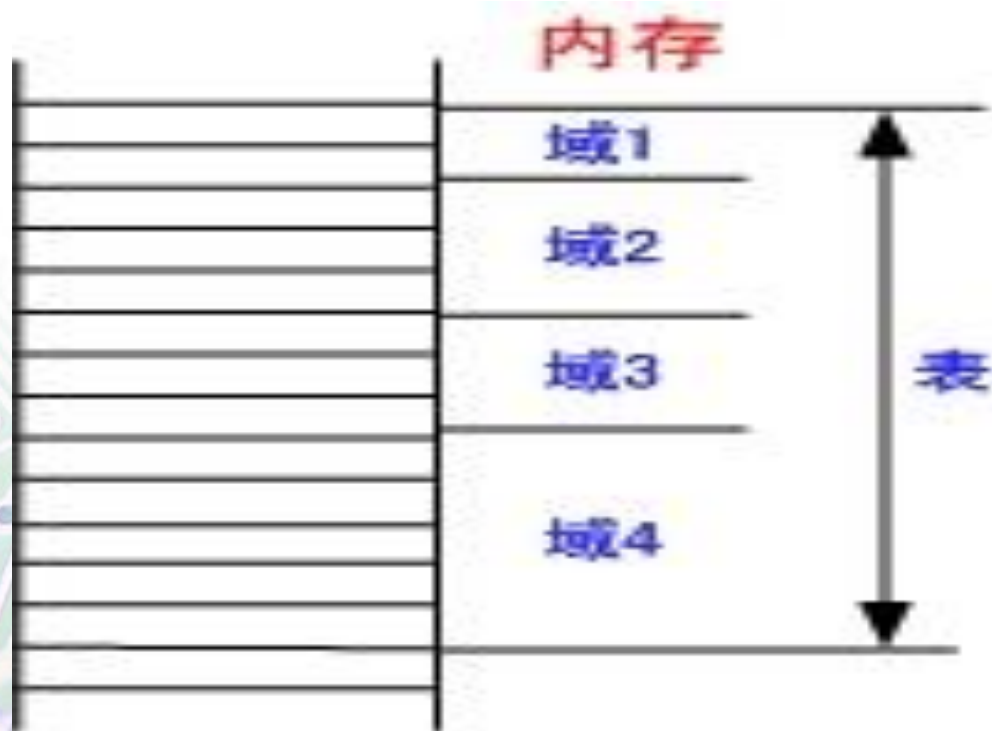
其目的是，防止在程序中使用LDR之类的指令访问时，可能产生的越界。

通常把数据缓冲池放在代码段的最后面，或放在无条件转移指令或子程序返回指令之后，这样处理器就不会错误地将数据缓冲池中的数据当作指令来执行。

4.2 通用伪指令

2、MAP 和 FIELD

在应用程序中经常使用一种如下图所示的表：



4.2 通用伪指令

MAP 用于定义一个结构化的内存表的首地址。MAP 可以用 “^” 代替。

语法格式：MAP <expr> {,<base_register>}

- expr是数字表达式或程序中的标号。当指令中没有base_register时，expr即为结构化内存表的首地址，可以为标号或数字表达式；
- base_register为基址寄存器（可选项）。当指令中包含这一项时，结构化内存表的首地址为expr与base_register寄存器值的和；

MAP fun ; fun就是内存表的首地址

MAP 0x100,R9 ; 内存表的首地址为R9+0X100

4.2 通用伪指令

MAP 通常和 FIELD 伪指令相配合来定义一个结构化的内存表。

FIELD 伪指令用于定义一个结构化内存表中的数据域。

指令格式：{label} FIELD expr

- Label为域标号，要顶格写；
- Expr表示本数据域在内存表中所占用的字节数；

功能：FIELD用于定义一个结构化内存表中的数据域，“#”与FIELD同义。

4.2 通用伪指令

MAP 0X100 ; 定义结构化内存表首地址为 0X100

A FIELD 16 ; 定义A的长度为16字节, 位置为 0X100

B FIELD 32 ; 定义B的长度为32字节, 位置为 0X110

S FIELD 256 ; 定义S的长度为256字节, 位置为 0X130

注意: MAP 和 FIELD 伪指令仅用于定义数据结构, 并不实际分配存储单元。FIELD 也可用 “#” 代替。

4.2 通用伪指令

3、SPACE

SPACE伪指令用于分配一片连续的存储区域并初始化为0。

指令格式: {label} SPACE expr

- label为内存块起始地址标号;
- Expr为所要分配的内存字节数;

SPACE 也可用 “%” 代替。

AREA DataRAM, DATA, READWRITE

; 声明一数据段, 名为DataRAM

DataSpace SPACE 100 ;分配连续的100字节的存储单元
;并初始化为 0。

4.2 通用伪指令

4、DCB

分配内存单元并初始化

指令格式: {label} DCB expr{, expr }{, expr }...

- label是存块起始地址标号;
- expr可以为0至255的数值或字符串, 内存分配的字节数由expr个数决定;

功能: DCB用于分配一段字节内存单元, 并用伪指令中的expr初始化, 一般可用来定义数据表格, 或文字字符串, “=” 与DCB同义。

4.2 通用伪指令

例如:

DISPTAB DCB 0x43,0x33,0x76,0x12

 DCB 120,20,32,44

String DCB "send,data is error!" ,0

 ; 构造字符串

使用示例:

LDR R1, =DISPTAB ;把DISPTAB的地址值送入R1

LDRB R2, [R1,#2] ;获取地址为[R1+#2]字节单元的值
 ;R2=0x76

4.2 通用伪指令

5、DCD和DCDU分配存储单元并初始化

指令格式: {label} DCD expr{, expr }{, expr }...

{label} DCDU expr{, expr }{, expr }...

- label是内存块起始地址标号
- expr为常数表达式或程序中标号, 内存分配字节数由expr个数决定

功能:

- DCD用于分配一段字内存单元, 并用伪指令中的expr初始化, 字对齐, 可定义数据表格或其它常数。 “&”与DCD同义。
- DCDU用于分配一段字内存单元, 并用伪指令中的expr初始化。DCDU伪指令分配的内存不需要字对齐, 可定义数据表格或其它常数。

4.2 通用伪指令

AREA blockcopy, CODE, READONLY

.....

LDR R1, =ftt

LDR R2, =ftt2

LDR R3, [R1]

LDR R4, [R2]

LDR R5, [R1, #4]

LDR R6, [R2, #4]

运算结果

; R3=1

; R4=3

; R5=2

; R6=4

.....

Src DCD 1,2,3,4,5,6,7,8,

MAP Src

ftt FIELD 8

ftt2 FIELD 8

END

该例说明了：MAP和FIELD伪指令不分配存储空间，只是给相关存储单元取个名称（标号）。便于程序以结构的方式访问对应的内存单元。

4.2 通用伪指令

7、DCFD和DCFDU (了解)

指令格式: {label}DCFD fpliteral{,fpliteral}{,fpliteral}...

{label}DCFDU fpliteral{,fpliteral}{,fpliteral}...

- label是内存块起始地址标号
- fpliteral是双精度的浮点数

功能:

- DCFD伪指令用于为双精度的浮点数分配一片连续的字存储单元并用伪指令中指定的表达式初始化。每个双精度的浮点数占据两个字单元。 DCFD分配的字存储单元是字对齐的。
- DCFDU具有DCFD同样的功能, 而用DCFDU分配的字存储单元并不严格字对齐。

4.2 通用伪指令

例如：

FdataTest DCFD 2E30, -3E-20

; 定义双精度浮点数, 字对齐

DCF_{DU} -0.1, 1000, 2.1E18

; 定义双精度浮点数, 字不对齐

DCFD伪指令分配的内存都是字对齐的，为了保证分配的内存都是字对齐，DCFD可能在分配的第一个内存单元插入填补字节。如何将双精度浮点数表达式转换成内存单元的内部表示形式是由浮点运算单元控制的。

4.2 通用伪指令

8、DCFS和DCFSU（了解）

指令格式：{label} DCFS fpliteral{,fpliteral}{,fpliteral}...

{label} DCFSU fpliteral{,fpliteral}{,fpliteral}...

- label是内存块起始地址标号
- fpliteral是单精度的浮点数

功能：

DCFS和DCFSU伪指令用于为单精度的浮点数分配一片连续的字存储单元并用伪指令中指定的表达式初始化。每个单精度的浮点数占据一个字单元，用DCFS分配的字存储单元是字对齐的，而用DCFSU分配的字存储单元并不严格字对齐。

4.2 通用伪指令

例如：

```
FdataTest    DCFS 1E3,-4E-9    ; 定义单精度浮点数,  
                                           ; 字对齐  
                DCFSU -0.1,3.1E6 ; 定义单精度浮点数,  
                                           ; 字不对齐
```

DCFS伪指令用于为单精度的浮点数分配字对齐的内存单元，为了保证分配的内存都是字对齐，DCFS可能在分配的第一个内存单元前插入添补字节。DCFSU分配的内存不需要字对齐。

4.2 通用伪指令

9、DCQ和DCQU (了解)

指令格式: {label}DCQ{-}literal{,literal}{,literal-}...

{label}DCQU{-}literal{,literal }{,literal-}...

- label是内存块起始地址标号
- literal是64位的数字表达式, 取值范围为 -2^{63} 到 $2^{63}-1$

功能:

- DCQ用于分配一段**双字的内存单元**, 并用**64位的整数数据**literal初始化, DCQ伪指令分配的内存需要字对齐。
- DCQU具有DCQ同样的功能, 但分配的内存不需要字对齐。

4.2 通用伪指令

AREA Miscdata,DATA,READONLY

Data DCQ -225,2101;

DCQU number+4 ; number必须是已经定义过的
; 数字表达式

DCQ伪指令可能在分配的**第一个内存单元**前插入多达3个字节的**填补字节**，以保证分配的内存是**字对齐**的。

DCQU分配的内存单元**不需要**字对齐。

4.2 通用伪指令

10、DCW和DCWU (了解)

指令格式: {label}DCW expr{,expr}{,expr }...

{label}DCWU expr{,expr }{,expr }...

- label是内存块起始地址标号
- expr 可以为程序标号或 数字表达式

功能:

- DCW用于分配一段半字的内存单元, 并用指定的数据expr初始化, DCW伪指令分配的内存需要半字对齐。
- DCWU具有DCW同样的功能, 但分配的内存不需要半字对齐。

4.2 通用伪指令

例如：

DCW -592, 123, 6543

DCW伪指令可能在分配的第一个内存单元前插入1字节的填补字节来保证分配的内存是半字节对齐的。DCWU分配的内存单元则不需要半字对齐。

4.2 通用伪指令

GBLA StrLen

GBLS Str1

GBLS Str2

GBLS Str3

Str1 SETS "AAA"

Str2 SETS "BBB"

Str3 SETS Str1 :CC: Str2 ;Str3= "AAABBB" (:CC:拼接运算符)

StrLenSETA :LEN: Str3 ; StrLen=6

; :LEN:是求字符串长度的运算符

AREA blockcopy,CODE,READONLY

.....

Src DCD 1,2,3,4,5,6,7,8

StrMem DCB Str3,0 ;从StrMem开始的连续7个单元

.....

END

;分别存放的是AAABBB的ASCII码和0.

4.2 通用伪指令

4.2.3 控制程序流向伪指令 (了解)

1. 条件汇编控制

IF、ELSE 和 ENDIF伪指令能根据条件的成立与否决定是否编译某个程序段。

IF 逻辑表达式

程序段1

ELSE

程序段2

ENDIF

4.2 通用伪指令

IF、ELSE、ENDIF 伪指令可以嵌套使用。

例

GBLL Test ; 声明一个全局逻辑变量Test

....

IF Test = TRUE

程序段1

ELSE

程序段2

ENDIF

4.2 通用伪指令

2、WHILE 和 WEND

WHILE 和 WEND 伪指令根据条件的成立与否决定是否重复汇编一个程序段。

WHILE 逻辑表达式

程序段

WEND

若WHILE后面的逻辑表达式为真，则重复汇编该程序段，直到逻辑表达式为假。

4.2 通用伪指令

WHILE 和 WEND 伪指令可以嵌套使用。

GBLA Counter; 声明一个全局数字变量Counter
Counter SETA 3; 赋值

.....

WHILE Counter < 10

程序段

WEND

4.2 通用伪指令

例：

```
        GBLA NU1
NU1     SETA 1
        AREA blockcopy, CODE, READONLY
        ENTRY
        WHILE NU1 < 3
        LDR R0, =Src
        LDR R1, =0x40000000
        MOV R2, #NU1
        LDR SP, =0x40000840
NU1     SETA NU1+1
        WEND
        .....
        END
```

4.2 通用伪指令

4.2.4 其他伪指令

1、定义对齐方式伪指令 **ALIGN**

指令格式：ALIGN {表达式, {偏移量}}

ALIGN是边界对齐伪指令，它可以通过添加填充字节的方式，使当前位置满足一定的对齐方式。其中表达式用于指定对齐方式在不同场合有不同的定义。

4.2 通用伪指令

对于在代码中单独使用的ALIGN伪指令，表达式的值，就是对齐方式的值，且该值必须是2的幂次方（2、4、8....）。

例

ALIGN 4 ; 4字节字对齐，ALIGN后面不能有等号。

比如：CODE16

thumbcode

ALIGN 4

CODE32

ARMCODE

4.2 通用伪指令

AREA OffsetExample, CODE

.....

ss1 DCB 1 ;假设ss1在0x01000字节

ALIGN 4,3 ; 4字节对齐+3偏移量.

ss2 DCB 1 ;

;使用 “ALIGN 4, 3” 以后,
;当前位置会转到0x01003(0x01000+3).
;ss1和ss2之间会空2个字节.

.....

4.2 通用伪指令

2、段定义伪指令AREA

AREA用于定义一个代码段或数据段。

指令格式：AREA sectionname {,attr} {,attr}...

- sectionname是定义的代码段或数据段的名称。若该名称是以数据开头的，则该名称必须用 “ | ” 括起来，如 | 2_datasec | 。还有一些代码段的名称是专有名称。
- Attr表示代码或数据段的属性，多个属性用短号分隔，常用的属性如下：

4.2 通用伪指令

属性	含义	备注
CODE	代码段	默认读/写属性为 READONLY
DATA	数据段	默认读/写属性为 READWRITE
NOINIT	数据段	指定此数据段仅仅保留了内存单元，而没有将各初始值写入内存单元。
READONLY	本段为只读	
READWRITE	本段为可读可写	
ALIGN表达式		ELF 的代码段和数据段为字对齐
COMMON	多源文件共享段	

4.2 通用伪指令

例：

```
AREA Init, CODE, READONLY
```

```
.....; 程序段
```

该伪指令定义了一个代码段，段名为 Init ，属性为只读。

一个汇编语言程序 **至少** 要有一个段。

4.2 通用伪指令

3、CODE16 和 CODE32

CODE16告诉汇编编译器后面的指令序列为16位的Thumb指令。

CODE32告诉汇编编译器后面的指令序列为32位的ARM指令。

语法格式：

CODE16

CODE32

注意：CODE16和CODE32只是告诉编译器后面指令的类型，该伪操作**本身不**进行程序状态的切换。

4.2 通用伪指令

例:

```
AREA    ChangeState, CODE, READONLY
```

```
ENTRY
```

```
CODE32                ;下面为32位ARM指令
```

```
LDR    R0,=start+1    ; 将跳转地址放入寄存器R0
```

```
BX     R0              ; 程序跳转到新的位置执行
```

```
.....                ; 并将处理器切换到Thumb工作状态
```

```
CODE16                ;下面为16位Thumb指令
```

```
start MOV    R1,#10
```

```
.....
```

```
END
```

4.2 通用伪指令

4、定义程序入口点伪指令 ENTRY

指定程序的入口点。

语法格式：

ENTRY

注意：一个程序（可包含多个源文件）中至少要有一个ENTRY（可以有多个ENTRY，当有多个ENTRY入口时，程序的真正入口点由链接器指定），但一个源文件中最多只能有一个ENTRY（可以没有ENTRY）

例

```
AREA Init, CODE, READONLY
```

```
ENTRY;
```

```
.....
```

4.2 通用伪指令

5、汇编结束伪指令 END

END 伪指令用于通知编译器汇编工作到此结束，不再往下汇编了。

语法格式：

END

例：

```
AREA Init, CODE, READONLY
```

```
...
```

```
END
```

注意：每一个汇编源程序都必须包含END伪操作，以表明本源程序的结束。

4.2 通用伪指令

6、外部可引用符号声明伪指令 EXPORT (或GLOBAL)

声明一个源文件中的符号，使此符号可以被其他源文件引用。

语法格式：

EXPORT/GLOBAL symbol {[weak]}

- symbol: 声明的符号的名称。（区分大小写）
- [weak]: 声明其他同名符号优先于本符号被引用。

例：

```
AREA example, CODE, READONLY
```

```
EXPORT DoAdd; 申明一个全局引用的标号DoAdd
```

```
DoAdd ADD R0, R0, R1
```

4.2 通用伪指令

7、IMPORT

当在一个源文件中需要使用另外一个源文件的外部可引用符号时，在被引用的符号前面，必须使用伪指令 IMPORT 对其进行声明：声明一个符号是在其他源文件中定义的。

语法格式：

```
IMPORT symbol{[weak]}
```

如果源文件声明了一个引用符号，则无论当前源文件中程序是否真正地使用了该符号，该符号均会被加入到当前源文件的符号表中。

- symbol: 声明的符号的名称。

4.2 通用伪指令

- [weak]:
 - 当没有指定此项时，如果symbol在所有的源文件中都没有被定义，则连接器会报告错误。
 - 当指定此项时，如果symbol在所有的源文件中都没有被定义，则连接器不会报告错误，而是进行下面的操作。
 - ✓ 如果该符号被B或者BL指令引用，则该符号被设置成下一条指令的地址，该B或BL指令相当于一条NOP指令。
 - ✓ 其他情况下此符号被设置成0。

例： AREA Init, CODE, READONLY
 IMPORT main
 ...
 END

4.2 通用伪指令

8、EXTERN

EXTERN 伪指令与 IMPORT 伪指令的功能基本相同，但如果当前源文件中的程序实际并未使用该符号，则该符号不会加入到当前源文件的符号表中。

其它与 IMPORT 相同。

4.2 通用伪指令

9、等效伪指令 EQU

EQU 伪指令用于为程序中的**常量**、**标号**等定义一个等效的**字符名字**，其作用类似于 C语言 中的 #define。

语法格式：name EQU expr{, type}

- name: 为expr定义的字符名称。
- expr: 基于寄存器的地址值、程序中的标号、32位的**地址常量**或者32位的**常量**。表达式，为**常量**。
- type: 当expr为**32位常量**时，可以使用type指示expr的**数据的类型**。取值为：
 - CODE32
 - CODE16
 - DATA

4.2 通用伪指令

例:

abcd EQU 2 ;定义abcd符号的值为2

abcd EQU label+16

;定义abcd符号的值为(label+16)

abcd EQU 0x1c, CODE32

;定义abcd符号的值为绝对地址0x1c,

;而且此处为ARM指令

4.2 通用伪指令

10、GET (或INCLUDE)

GET 伪指令用于将一个源文件包含到当前的源文件中，并将被包含的源文件在当前位置进行汇编。

语法格式：GET 文件名

可以使用 INCLUDE 代替 GET。

GET 伪指令只能用于包含源文件，包含目标文件则需要使用 INCBIN 伪指令。

4.2 通用伪指令

例：

AERA Init, CODE, READONLY

GET a1.s

GET c:\a2.s

...

END

4.2 通用伪指令

11、INCBIN (了解)

INCBIN 伪指令用于将一个目标文件或数据文件包含到当前的源文件中，被包含的文件不做任何变动地存放在当前文件中，编译器从其后开始继续处理。

语法格式：INCBIN 文件名

例：

```
AREA Init, CODE, READONLY
```

```
INCBIN a1.dat
```

```
INCBIN c:\a2.txt
```

4.3 与ARM指令相关的宏指令

4.3.1 宏

1、MACRO 和 MEND

MACRO 和 MEND 伪指令可以为一个程序段定义一个名称。这样，在汇编语言应用程序中，就可以通过这个名称来使用它所代表的程序段，即当程序做汇编时，该名称将被替换为其所代表的程序段。

4.3 与ARM指令相关的宏指令

MACRO

\$标号 宏名 \$参数1, \$参数2,

程序段 (宏定义体)

MEND

- \$标号：为主标号，宏内的所有其它标号必须由主标号组成；
- 宏名：宏名称，为宏在程序中的引用名；
- \$参数1, \$参数2：宏中可以使用的参数。

宏中的所有标号必须在前面冠以符号 “\$” 。

MACRO、MEND伪指令可以嵌套使用。

4.3 与ARM指令相关的宏指令

例：

MACRO ; 宏定义指令

\$MDATA MAXNUM \$NUM1,\$NUM2 ; 主标号, 宏名, 参数

语句段

\$MDATA.WAY1 ; 宏内标号, 必须写为 “主标号.宏内标号”

语句段

\$MDATA.WAY2 ; 宏内标号

语句段

MEND ; 宏结束指令

主程序中调用该宏：

Lab1 MAXNUM 0x01, 0x02

4.3 与ARM指令相关的宏指令

2、MEXIT

MEXIT 用于从宏定义中跳转出去。

语法格式：MEXIT

4.3 与ARM指令相关的宏指令

例：该宏是用于中断服务程序的一个宏定义，除reset中断外的其余6种中断，都可以使用该宏定义的程序段进入。

MACRO

\$HandlerLabel HANDLER \$HandleLabel;宏定义

\$HandlerLabel

sub sp,sp,#4 ; ATPCS规定ARM数据堆栈为FD

; (满递减) 型堆栈。

; 这里将堆栈退一个字预留

; 用于保存跳转地址

stmfd sp!,{r0} ;将R0压入堆栈

4.3 与ARM指令相关的宏指令

`ldr r0,=$HandleLabel` ;将HandleLabel的
;地址赋给R0

`ldr r0,[r0]` ;将HandleLabel的地址指向的内容
;(实际的执行地址)赋给R0

`str r0,[sp,#4]`;将对应的中断函数首地址入栈保护

`ldmfd sp!,{r0,pc}` ;将中断函数的首地址出栈,
;放入PC中, 系统将跳转到对
;应中断处理函数

MEND

4.3 与ARM指令相关的宏指令

在程序中调用该宏

HandlerFIQ HANDLER HandleFIQ

;通过宏的名称HANDLER调用宏,

;其中宏的标号为HandlerFIQ, 参数为HandleFIQ

HandlerIRQ HANDLER HandleIRQ

HandlerUndef HANDLER HandleUndef

HandlerSWI HANDLER HandleSWI

HandlerDabort HANDLER HandleDabort

HandlerPabort HANDLER HandlePabort

4.3 与ARM指令相关的宏指令

例如：HandlerFIQ HANDLER HandleFIQ展开后为：

HandlerFIQ ; HandlerFIQ 替换了主标号\$HandlerLabel

sub sp,sp,#4

stmfd sp!,{r0}

ldr r0,=HandleFIQ

; HandleFIQ替换了参数,=\$HandlerLabel

ldr r0,[r0]

str r0,[sp,#4]

ldmfd sp!,{r0,pc}

4.3 与ARM指令相关的宏指令

已知某定义的宏，使用语句 `HandlerFIQ HANDLER` `HandleFIQ`调用该宏，在程序预编译后，该宏调用语句将展开为以下程序段：

`HandlerFIQ`

```
sub sp,sp,#4
stmfd sp!,{r0}
ldr r0,=HandleFIQ
ldr r0,[r0]
str r0,[sp,#4]
ldmfd sp!,{r0,pc}
```

请写出该宏的定义程序段。（该题答案见前面宏定义示例。）

解题注意事项：

务必满足宏定义的格式（`MACRO`、`MEND`），
正确的定义主标号和参数。

4.3 与ARM指令相关的宏指令

4.2.2 宏指令

在ARM中，还有一种汇编器**内置的**无参数和标号的宏——**宏指令**。

在汇编时，这些**宏指令**被**替换成**一条或两条**真正的**ARM或 Thumb 指令。ARM宏指令有四条，分别是：

- ADR：小范围的地址读取宏指令；
- ADRL：中等范围的地址读取宏指令；
- LDR：大范围的地址读取宏指令；
- NOP：空操作宏指令。

4.3 与ARM指令相关的宏指令

1、小范围的地址读取宏指令ADR

ADR 指令用于将一个 **近地址**值 传递到一个寄存器中。

指令格式: ADR{cond} <reg>, <expr>

- reg 为目标寄存器名称;
- expr 为表达式。该表达式通常是程序中一个表示**存储位置**的 **地址标号**。

该宏指令的功能是把**标号**所表示的**地址**传递到目标寄存器中。

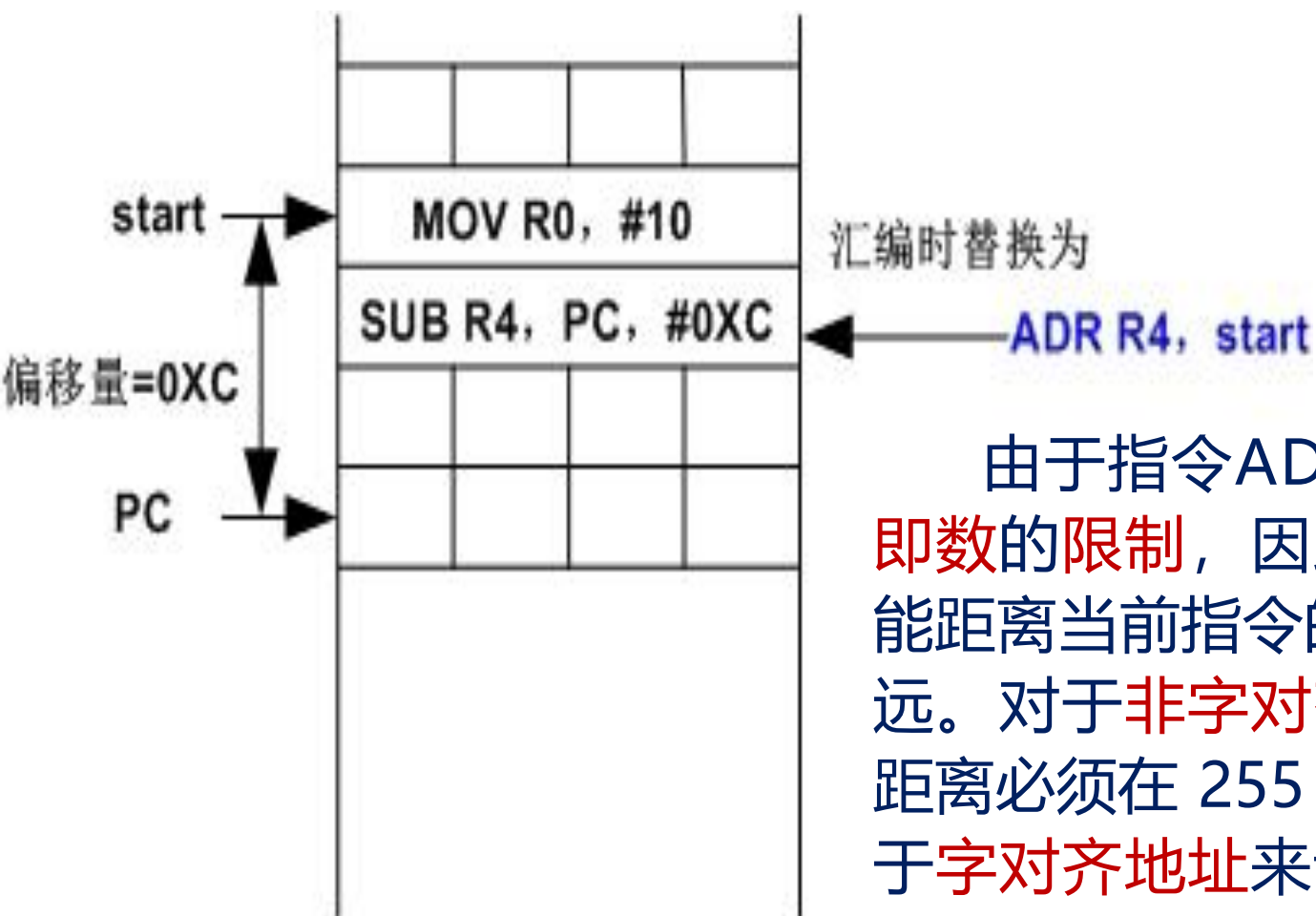
4.3 与ARM指令相关的宏指令

汇编器在汇编时，将把ADR宏指令替换成一条真正的ADD或SUB指令，以当前的PC值减去或加上expr与PC之间的偏移量得到标号的地址，并将其传递到目标寄存器。若不能用一条指令实现，则产生错误，编译失败。

例：

```
start MOV R0,#10  
      ADR R4,start
```

4.3 与ARM指令相关的宏指令



由于指令ADD或SUB中对**立即数**的限制，因此**标号地址**就不能距离当前指令的地址（PC）过远。对于**非字对齐地址**来说，其距离必须在 255 字节以内；而对于**字对齐地址**来说，距离必须在 1020 字节以内。所以ADR也叫做**近地址**读取指令。

4.3 与ARM指令相关的宏指令

例2：查表

ADR R0,D_TAB ;加载转换表地址

LDRB R1,[R0,R2] ;使用R2作为参数，进行查表

.....

D_TAB

DCB 0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92

4.3 与ARM指令相关的宏指令

2、中等范围的地址读取宏指令ADRL

类似于ADR，但可以把更远的地址赋给目标寄存器。

指令格式：ADRL{cond} <reg>, <expr>

- reg 为目标寄存器名称；
- Expr 为表达式，必须是64KB以内非字对齐地址，或256KB以内的字对齐地址。

该指令只能在ARM状态下使用，在Thumb状态下不能使用。汇编时，ADRL宏指令由汇编器替换成两条合适的指令。

4.3 与ARM指令相关的宏指令

例:

```
start      MOV R0,#10
```

```
          ADRL R4,start + 60000
```

此时, start为PC-12

则 $\text{start} + 60000 = \text{PC} + 59988$ (**0XEA54**)

其中 ADRL 将被替换为如下两条指令:

```
          ADD R4, PC, #0XE800
```

```
          ADD R4, R4, #0X254
```

如果汇编器找不到合适的两条指令, 将会报错。

4.3 与ARM指令相关的宏指令

例:

ADRL伪指令举例如下:

ADRL R0, DATA_BUF

.....

ADRL R1, DATA_BUF +80

.....

DATA_BUF

SPACE 100 ;定义100字节缓冲区

4.3 与ARM指令相关的宏指令

3、大范围的地址读取宏指令LDR

指令格式：LDR{cond} reg,={expr | label - expr}

- reg: 目标寄存器名称;
- expr: 32位常数;
- label - expr: 为地址表达式。

程序经常用这条指令把一个地址传递到寄存器reg中。汇编器在对这种指令进行汇编时，会根据指令中expr的**值的大小**来把这条指令替换为**合适**的指令。

与ARM指令的LDR的区别：伪指令LDR的参数有“=”号。

4.3 与ARM指令相关的宏指令

- ① 当 `expr` 的值未超过 `MOV` 或 `MVN` 指令所限定的取值范围时，汇编器用 ARM 的 `MOV` 或 `MVN` 指令来取代宏指令 `LDR`;
- ② 当 `expr` 的值超过 `MOV` 或 `MVN` 指令所限定的取值范围时，汇编器将常数 `expr` 放在由 `LTORG` 定义的文字缓冲池，同时用一条 ARM 的装载指令 `LDR` 来取代宏指令 `LDR`，而这条装载 `LDR` 指令则用 `PC` 加偏移量的方法到文字缓冲池中把该常数读取到指令指定的寄存器。

由于这种指令可以传递一个 32 位地址，因此也被叫做全范围地址读取指令。

4.3 与ARM指令相关的宏指令

例4:

```
.....  
(0x060) LDR R1,=Delay  
.....  
Delay  
(0x102) MOV R0,R14  
.....
```

使用 LDR 将程序标号
Delay所表示的地址存入
R1。

编译后的对应的代码:

```
.....  
LDR R1,stack  
.....  
Delay  
MOV R0,R14  
.....
```

```
LTORG  
stack DCD 0x102
```

LDR伪指令被汇编成一条
LDR指令,并在文字池中定义
一个常量,该常量为标号
Delay的地址。

4.3 与ARM指令相关的宏指令

例:

LDR R0, =0x12345678 ;加载32位立即数0x12345678

LDR R0, =DATA_BUF + 60 ;加载DATA_BUF地址+60

.....

LTORG

.....

4.3 与ARM指令相关的宏指令

4、NOP

汇编器对NOP指令进行汇编时，会将其转换为：

MOV R0, R0

指令格式：NOP

例：延时子程序

Delay

NOP ;空操作

NOP

SUBS R1,R1,#1 ;循环次数减1

BNE Delay

MOV PC,LR

程序举例

AREA example, CODE, READONLY ;伪指令操作示例1

ENTRY

MOV R0,#1

B start

DATA1 DCB "strin"

ALIGN 4

;造成地址不对齐

;使后续指令4字节对齐

start

BL func1

BL func2

B start

程序举例

func1

LDR r0, =start

LDR r1, =Darea +12

LDR r2, =Darea + 6000

MOV pc,lr ;返回

LTORG ;子程序结束, 声明文字池

func2

LDR r3, =Darea +60

LDR r4, =Darea +6004

MOV pc, lr

Darea SPACE 8000

END

程序举例

AREA example, CODE, READONLY

;伪指令操作示例2

ENTRY

CODE32

START CMP R0, #8

ADDLT PC, PC, R0, LSL#2

B method_d

B method_0

B method_1

B method_2

B method_3

B method_4

B method_5

B method_6

B method_7

程序举例

method_0

MOV R0,#1
B end0

;这里添加实现method_0的代码

method_1

MOV R0,#2
B end0

;这里添加实现method_1的代码

.....

.....

method_7

MOV R0,#8
B end0

;这里添加实现method_7的代码

method_d MOV R0,#0

end0 B START

END

ARM汇编程序设计模板要点

AREA example, CODE, READONLY

;定义代码段, 一个ARM汇编程序至少有一个代码段

ENTRY ;定义程序入口点

.....

.....

END ;汇编语言程序结束

4.4 汇编语言编程规范

4.4.1 汇编语言的语句格式

ARM (Thumb) 汇编语言的语句格式为：

{<标号>} <指令或伪指令> {; 注释}

在汇编语言程序设计中，每一条指令的助记符可以全部用大写或全部用小写，但不允许在一条指令中大小写混用。**标号要顶格。**

如果一条语句太长，则可将该长语句分成若干行来书写，每行的末尾用“\”来表示下一行与本行为同一条语句。但是注意：在“\”之后不能再有其他字符,包括空格和制表符。

4.4 汇编语言编程规范

4.4.2 汇编语言程序中常用的符号

在ARM汇编语言中，符号可以代表地址、变量和数字常量。

符号命名规则如下：

- ① 符号由大小写字母、数字以及下画线组成；
- ② 除局部标号以数字开头，其他的符号都不能以数字开头；
- ③ 符号是区分大小写的；
- ④ 符号在其作用范围内必须唯一；
- ⑤ 程序中的符号不能与指令助记符、伪指令、宏指令同名。

这些规则主要涉及程序中的变量和常量。

4.4 汇编语言编程规范

4.4.3 汇编语言的表达式和运算符

1、运算次序遵循如下的优先级：

- (1) 优先级相同的双目运算符运算顺序为从左到右；
- (2) 相邻的单目运算符的运算顺序为从右到左，且单目运算符的优先级高于其他运算符；
- (3) 括号运算符的优先级最高。

4.4 汇编语言编程规范

2、数字表达式及运算符

(1) +、-、*、/及MOD算术运算符

$X + Y$ 表示 X 与 Y 的和;

$X - Y$ 表示 X 与 Y 的差;

$X * Y$ 表示 X 与 Y 的乘积;

X / Y 表示 X 除以 Y 的商;

$X : MOD: Y$ 表示 X 除以 Y 的余数。

4.4 汇编语言编程规范

(2) ROL、ROR、SHL及SHR移位运算符

X :ROL: Y 表示将X循环左移Y位;

X :ROR: Y 表示将X循环右移Y位;

X :SHL: Y 表示将X左移Y位;

X :SHR: Y 表示将X右移Y位。

4.4 汇编语言编程规范

3) AND、OR、NOT及EOR按位逻辑运算符

X :AND: Y 表示将X和Y按位做逻辑“与”的操作。

X :OR: Y 表示将X和Y按位做逻辑“或”的操作；

:NOT: Y 表示将Y按位做逻辑“非”的操作；

X :EOR: Y 表示将X和Y按位做逻辑“异或”的操作。

4.4 汇编语言编程规范

3、逻辑表达式及运算符

(1) =、>、<、>=、<=、/=、<>运算符

$X = Y$ 表示X等于Y;

$X > Y$ 表示X大于Y;

$X < Y$ 表示X小于Y;

$X \geq Y$ 表示X大于或等于Y;

$X \leq Y$ 表示X小于或等于Y;

$X \neq Y$ 表示X不等于Y;

$X \lt \gt Y$ 表示X不等于Y。

4.4 汇编语言编程规范

(2) LAND、LOR、LNOT及LEOR运算符

X :LAND: Y 表示将X和Y做逻辑 “与” 的操作;

X :LOR: Y 表示将X和Y做逻辑 “或” 的操作;

:LNOT: Y 表示将Y做逻辑 “非” 的操作;

X :LEOR: Y 表示将X和Y做逻辑 “异或” 的操作。

4.4 汇编语言编程规范

4、字符串表达式及运算符

编译器所支持的字符串最大长度为512 字节。

(1) LEN 运算符

LEN 运算符返回字符串的长度（字符数），以 X 表示字符串表达式。

:LEN: X

示例:

```
GBLS STR
GBLA LEN
STR SETS "AAA"
LEN SETA :LEN:STR          ;LEN=3
```

4.4 汇编语言编程规范

(2) CHR 运算符

CHR 运算符将 0~255 之间的整数转换为一个字符，以 M 表示一个整数，其格式如下：

:CHR: M

在内存单元中，其值还是没有变。只是经过这种处理后，可以把它当做一个字符，用于字符串处理函数中。

4.4 汇编语言编程规范

(3) STR 运算符

STR 运算符将一个数字表达式或逻辑表达式转换为一个字符串。

对于 数字表达式，STR 运算符将其转换为一个以 十六进制组成的字符串；对于 逻辑表达式，STR 运算符将其转换为 字符串"T"或"F"。

:STR: X ; X为数字表达式或逻辑表达式

举例:

GLBA A1

A1 SETA 15

:STR:A1 ;将A1转换为" 0000000F"

4.4 汇编语言编程规范

(3) STR 运算符

例2:

```
AREA blockcopy, CODE, READONLY
CODE32
LDR R0, =TTQ
LDR R1, TTQ
LDR R4, [R0]
TTQ DCB :STR: 0x12345678
END
```

该程序执行后，R1的值为0x34333231（十六进制），
PS：0x34是4对应的ASCII码，0x31是1对应的ASCII码。

4.4 汇编语言编程规范

(4) LEFT 运算符

LEFT 运算符返回某个字符串 左端 的一个子集。

$X : \text{LEFT} : Y$

X 为源字符串, Y 为一个整数, 表示要返回的字符个数。

举例:

```
GBLS STR1
```

```
GBLS STR2
```

```
STR1 SETS "AAABBB"
```

```
STR2 SETS STR1 :LEFT:3
```

程序运行完后,STR2为" AAA"

4.4 汇编语言编程规范

(5) RIGHT 运算符

RIGHT 运算符返回某个字符串 右端 的一个子集。

$X : \text{RIGHT} : Y$

X 为源字符串, Y 为一个整数, 表示要返回的字符个数。

举例:

```
GBLS STR1
```

```
GBLS STR2
```

```
STR1 SETS "AAABBB"
```

```
STR2 SETS STR1 : RIGHT : 3
```

程序运行完后,STR2为" BBB"

4.4 汇编语言编程规范

(6) CC 运算符

CC 运算符用于将两个字符串连接成一个字符串。

$X : CC : Y$

X为源字符串1，Y为源字符串2，CC运算符将Y连接到X的后面。

举例:

```
GBLS STR1;声明字符串变量STR1
```

```
GBLS STR2;声明字符串变量STR2
```

```
STR1 SETS "AAACCC" ;变量STR1赋值为" AAACCC"
```

```
STR2 SETS "BBB" :CC:(STR1:LEFT:3)
```

程序运行完后,STR2为" BBBAAA"

4.4 汇编语言编程规范

(6) CC 运算符

举例:

GBLL LL1

GBLS GS1

GBLS GS2

LL1 SETL {TRUE}

GS1 SETS "ABCD"

GS2 SETS :STR:LL1:CC::CHR:65

:STR:LL1 = "T"

:CHR: 65 = "A"

GS2= "TA"

4.4 汇编语言编程规范

8、程序中的变量代换

程序中的变量可通过代换操作取得一个常量。代换操作符为“\$”。

如果在数字变量前面有一个代换操作符“\$”，则编译器会将该数字变量的值转换为十六进制的字符串，并将该十六进制的字符串代换“\$”后的数字变量。

如果在逻辑变量前面有一个代换操作符“\$”，则编译器会将该逻辑变量代换为它的取值（真或假）。

如果在字符串变量前面有一个代换操作符“\$”，则编译器会将该字符串变量的值代换“\$”后的字符串变量。

4.4 汇编语言编程规范

例：

GBLS s1 ; 定义局部字符串变量S1和S2

GBLS s2

s1 SETS "Test !"

s2 SETS "This is a \$s1"

字符串变量S2的值为 "This is a Test !"

4.4 汇编语言编程规范

例：

GBLA Test1

GBLL Test2

GBLL Test3

Test1 SETA 0x66

Test2 SETL {TRUE}

Test3 SETL {FALSE}

StrMem DCB "12345\$Test1\$Test2\$Test3 "

StrMem存放的内容为字符串： “1234500000066TF” 对应的
ASCII码

课堂练习

```
AREA blockcopy, CODE, READONLY
```

```
ENTRY
```

```
LDR R1, =ftt
```

```
LDR R2, =ftt2
```

```
LDR R3, [R1]
```

```
LDR R4, [R2]
```

```
LDR R5, [R1, #4]
```

```
LDR R6, [R2, #4]
```

```
Src DCD 1, 2, 3, 4, 5, 6, 7, 8, 1, 2, 3, 4, 5, 6, 7, 8, 1, 2, 3, 4
```

```
StrMem DCB "ABCS", 0
```

```
MAP Src
```

```
ftt FIELD 8
```

```
ftt2 FIELD 8
```

```
END
```

该程序段执行后，R3=**1**，R4=**3**，R5=**2**，R6=**4**。

课堂练习

```
AREA blockcopy, CODE, READONLY
```

```
ENTRY
```

```
ARM
```

```
LDR R0, =ss1
```

```
LDR R1, =ss2
```

```
LDR R4, [R0]
```

```
ALIGN 4
```

```
ss1 DCB 1
```

```
ALIGN 4, 3
```

```
ss2 DCB 3
```

```
END
```

该段程序执行完成后，假如R0的值为0x10001000，
则R1=0x10001003，R4=0x03000001

课堂练习

```
GBLL 1111
GBLS Str1
GBLS Str2
GBLS Str3
GBLS Str4
Str1 SETS "AAAA"
Str2 SETS "BCDEFG"
1111 SETL {False}
Str3 SETS Str1 :CC: (Str2:LEFT:3):CC: :STR: 1111
Str4 SETS Str1 :CC: Str2:LEFT:3:CC: :STR: 1111
```

Diagram illustrating string concatenation and substring extraction:

- Red box around `(Str2:LEFT:3)` in Str3 points to **BCD**.
- Red box around `:STR: 1111` in Str3 points to **F**.
- Red box around `Str1 :CC: Str2:LEFT:3` in Str4 points to **AAAABCDEF**.
- Blue box around `:STR: 1111` in Str4 points to **AAA**.

该段程段中Str3和Str4对应的字符串分别为:

Str3对应的字符串为: **AAAABCDF**

Str4对应的字符串为: **AAAF**

4.5 ARM工程

由于C语言便于理解，有大量的支持库，所以它是当前 ARM 程序设计所使用的**主要编程语言**。

对硬件系统的初始化、CPU状态设定、中断使能、主频设定以及RAM控制参数初始化等C程序力所不能及的**底层操作**，还是要由**汇编语言程序**来完成。

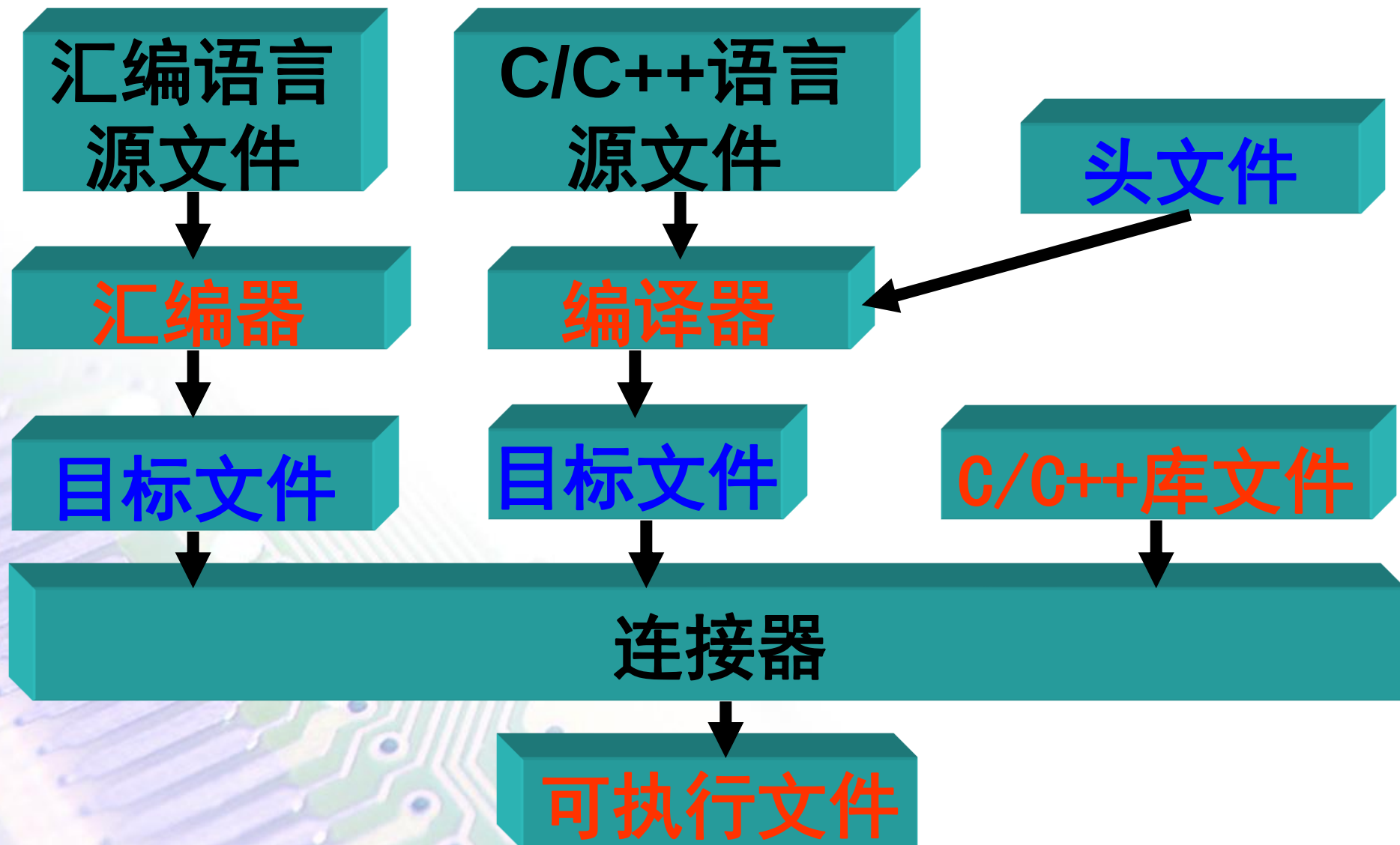
4.5 ARM工程

用汇编语言或C/C++语言编写的程序叫做源程序，对应的文件叫做源文件。

一个ARM工程应由多个文件组成，其中包括扩展名为 .S 的汇编语言源文件、扩展名为 .C的C语言源文件，扩展名为 .CPP的C++源文件、扩展名为.H的头文件等。

ARM工程的各种源文件之间的关系，以及最后形成可执行文件的过程如下：

4.5 ARM工程



4.5 ARM工程

ARM 提供的开发工具Code Warrior for ARM 中包含的编译器如下：

编译器	语言种类	源文件类型	源文件扩展名	目标文件类型
Armcc	C	C	. c	ARM代码
Tcc	C	C	. c	Thumb代码
Armcpp	C++	c/c++	. c/. cpp	ARM代码
tcpp	C++	c/c++	. c/. cpp	Thumb代码

4.5 ARM工程

除了C和C++编译器，Code Warrior for ARM 开发工具还提供了汇编器ARMASM。

编译器负责生成目标文件，它是一种包含了调试信息的ELF格式文件。

ELF(Executable and Linking Format): 可执行连接格式。

可执行连接格式是UNIX系统实验室(USL)作为应用程序二进制接口(Application Binary Interface(ABI)而开发和发布的。

4.5 ARM工程

编译器还要生成列表文件等相关文件：

文件扩展名	说明
.h	头文件
.o	<u>ELF</u> 格式的目标文件
.s	汇编代码文件
.lst	错误及警告信息列表文件

4.5 ARM工程

各种源文件先由编译器和汇编器将它们分别编译或汇编成汇编语言文件及目标文件。

连接器负责将所有目标文件连接成一个文件并确定各指令的确定地址，从而形成最终可执行文件。

连接器有 三个 功能：

- ① 根据程序员所指定的选项，为程序分配地址空间；
- ② 生成与地址相关的代码，把所有文件连接成一个可执行文件；
- ③ 给出连接信息，以说明连接过程和连接结果。

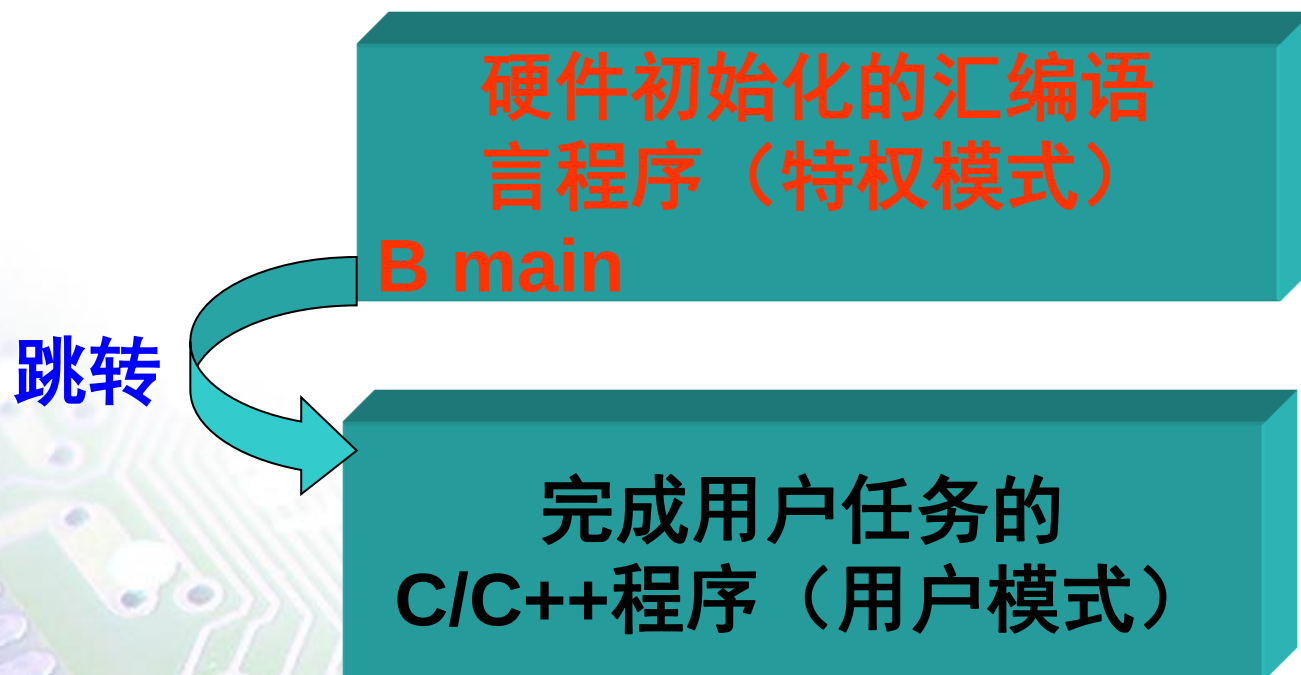
4.6 ARM程序框架

在应用系统的程序设计中，若所有的编程任务均用汇编语言来完成，其工作量是可想而知的，这样做也不利于系统升级或应用软件移植。

通常汇编语言部分完成系统硬件的初始化；高级语言部分完成用户的应用。

执行时，首先执行初始化部分，然后再跳转到 C/C++ 部分。整个程序结构显得清晰明了，容易理解。程序的基本结构如下：

4.6 ARM程序框架



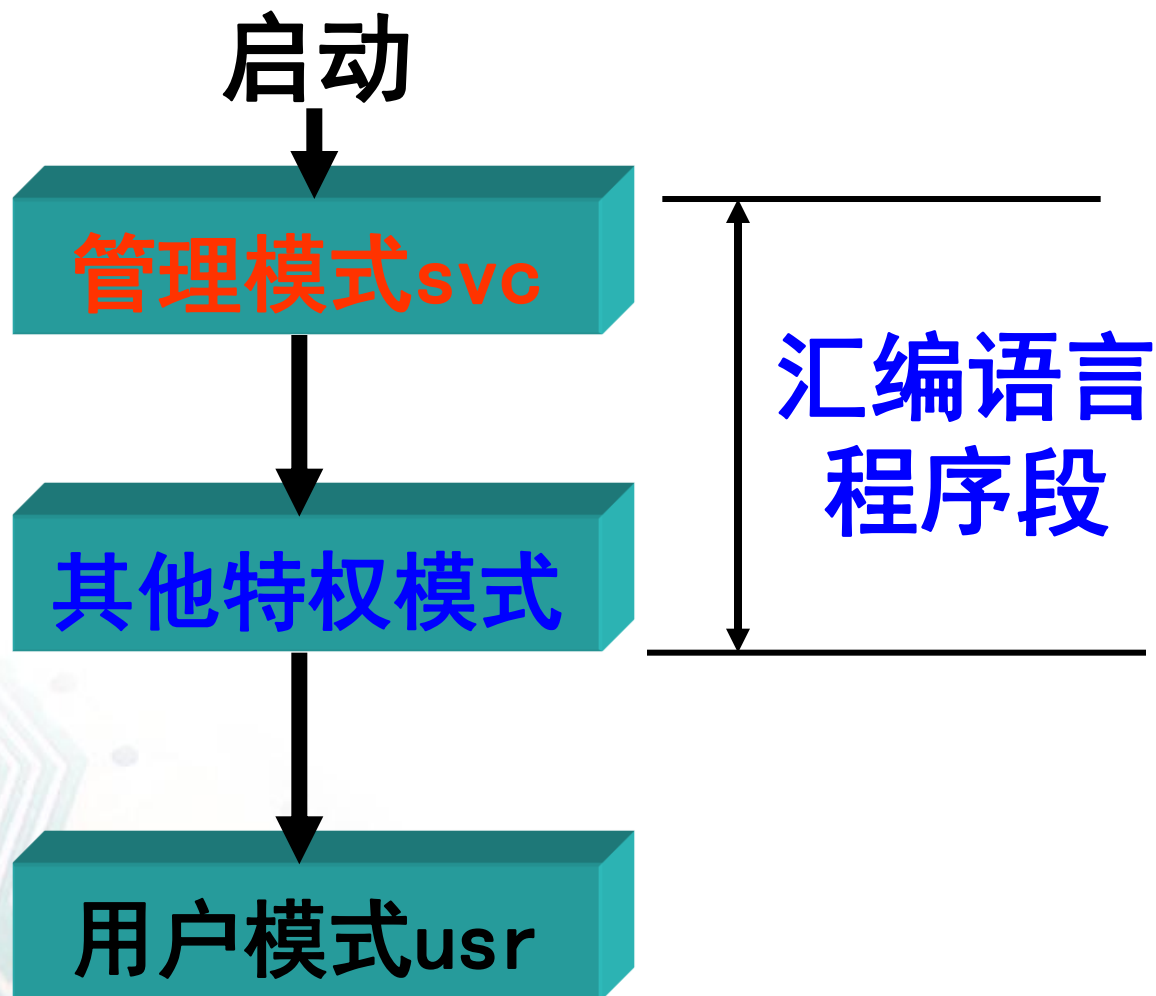
4.6 ARM程序框架

4.6.1 初始化程序部分

由于在用于完成初始化任务的汇编语言程序中需要在特权模式下做一些诸如修改 CPSR 等特权操作，所以不能过早地进入用户模式。（上电复位后进入 SVC 模式，除用户模式外，其它模式中能任意切换工作模式）

4.6 ARM程序框架

通常，初始化过程大致会经历如下所示的一些模式变化。



4.6 ARM程序框架

4.6.2 初始化部分与主应用程序部分的衔接

当所有的系统初始化工作完成之后，就需要把程序流程转入主应用程序。最简单的方法是，在汇编语言程序末尾使用跳转指令 **B** 或 **BL** 直接从启动代码转移到 C/C++ 程序入口。

B main ;跳转到C/C++程序

同时在汇编文件中有如下代码：

IMPORT main

4.6 ARM程序框架

完整的汇编语言程序大致如下：

```
IMPORT  main
AREA  Init, CODE, READONLY
ENTRY
LDR SP, =0X3EE1000
B main
END
```

C程序如下：

```
void  main(void)
{
    ....
}
```

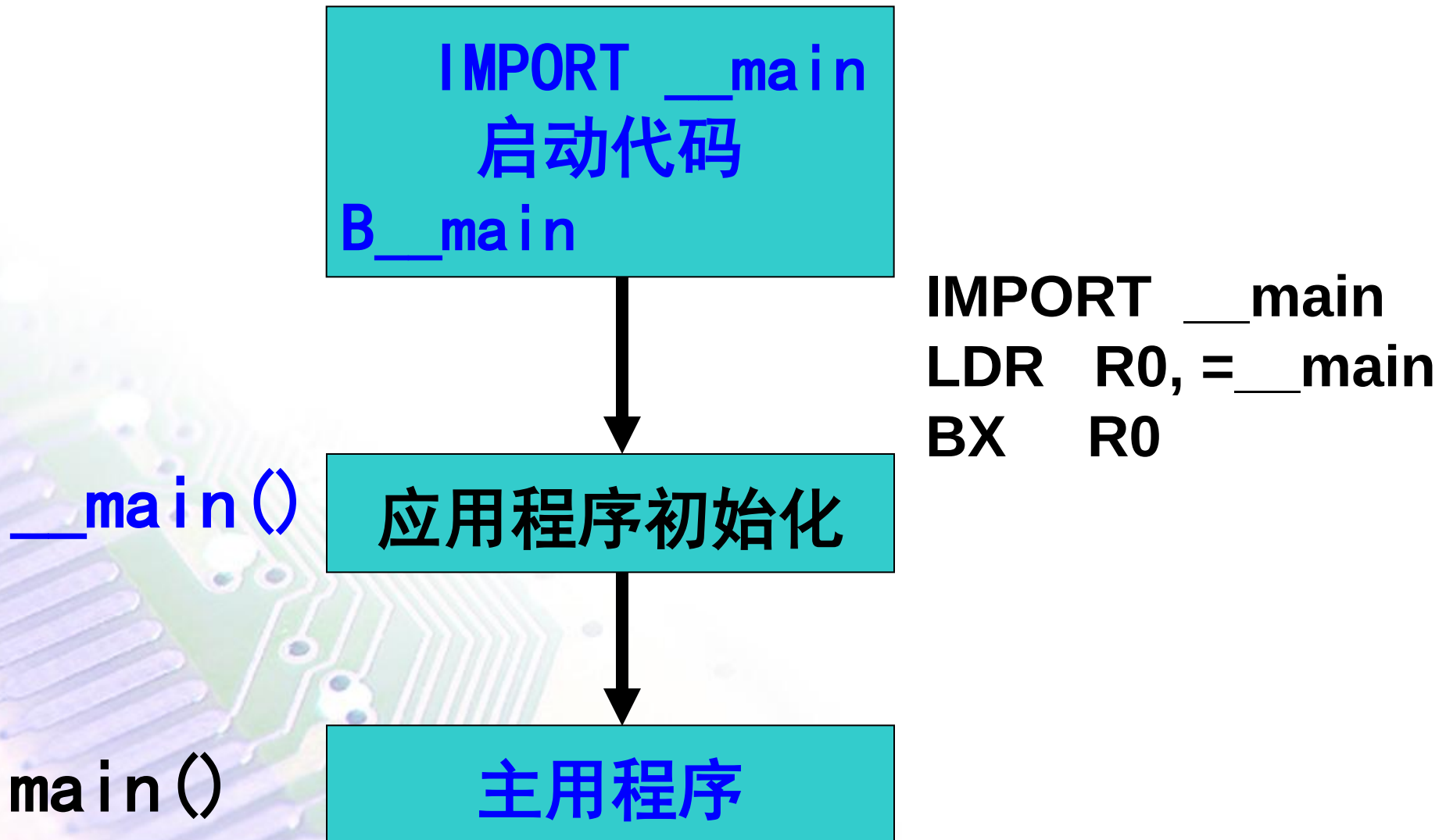
4.6 ARM程序框架

4.6.3 ARM开发环境提供的程序框架

为方便工程开发，ARM 公司的开发环境ARM ADS 为用户提供了一个可以选用的应用程序框架。该框架把为用户程序做准备工作的程序分成了：**启动代码**和**应用程序初始化**两部分。

用于**硬件初始化的汇编语言**部分叫做**启动代码**；用于**应用程序初始化的C部分**叫做**初始化部分**。整个程序如下所示：

4.6 ARM程序框架



4.7 ARM汇编语言程序设计

4.7.1 段

汇编语言编写的程序叫做汇编语言源程序，包含源程序的文件叫做汇编语言程序文件。

一个工程可以有多个源文件，汇编源文件的扩展名为 .S。

在ARM (Thumb) 汇编语言程序中，通常以段为单位来组织代码。段是具有特定名称且功能相对独立的指令或数据序列。

根据段的内容，分为代码段和数据段。

一个汇编程序至少应该有一个代码段，当程序较长时，可以分割为多个代码段和数据段。

4.7 ARM汇编语言程序设计

以下是一个汇编语言程序段的基本结构：

```
        AREA Init, CODE, READONLY    ; 只读的代码段Init
        ENTRY                          ; 程序入口点

start   LDR R0, =0X3FF5000
        LDR R1, =0XFF    ; 或MOV R1, #0XFF
        STR R1, [R0]
        LDR R0, =0X3FF5008
        LDR R1, =0X01    ; 或MOV R1, #0X01
        STR R1, [R0]
        .....
        END                                ; 段结束
```

4.7 ARM汇编语言程序设计

```
DataMem    EQU 0x40000000
            ; 定义一个符号常量DataMem其值为0x40000000

SPAddr     EQU 0x40000FF0

GBLA StrLen ; 定义一个全局数字变量StrLen
GBLS Str1   ; 定义一个全局字符串变量Str1
GBLS Str2
GBLS Str3

Str1       SETS "AAAA" ; 给全局字符串变量Str1赋值“AAAA”
Str2       SETS "BBBB"
Str3       SETS Str1 :CC: Str2
StrLen     SETA :LEN: Str3   StrLen全局数值变量的值为: 8

AREA blockcopy, CODE, READONLY ; 段定义
ENTRY                          ; 入口点

LDR R0, =StrMem
; 使用LDR伪指令把值StrMem赋值给R0寄存器
LDR R1, =DataMem
LDR SP, =SPAddr
MOV R2, #StrLen
```

4.7 ARM汇编语言程序设计

MOV R2, R2, LSR #2

OneCopy LDR R3, [r0], #4
 STR R3, [r1], #4
 SUBS R2, R2, #1
 BNE OneCopy

Stop B Stop

LTORG ;定义数据缓冲池

StrMem DCB Str3, 0 ;分配内存单元并初始化为Str3的值
 ALIGN 4 ;定义后续存储单元以字对齐方式分配
Temp SPACE 100 ;分配100字节连续的字节单元

END

4.7 ARM汇编语言程序设计

4.7.2 分支程序设计

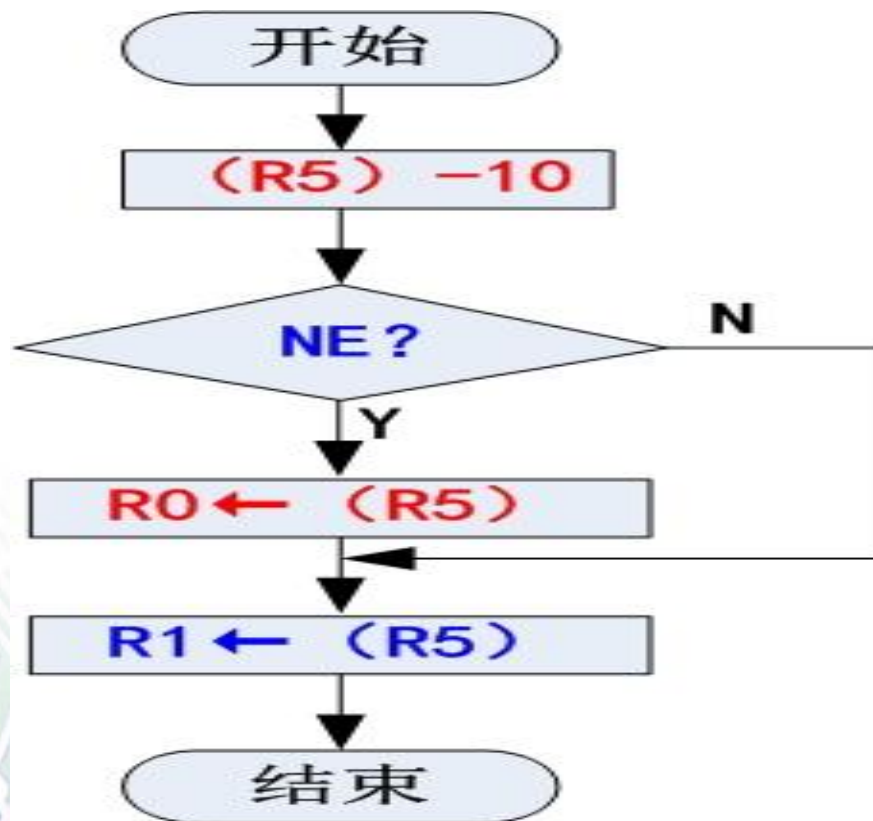
具有两个或两个以上可选执行路径的程序叫做分支程序。

1、普通分支程序设计

使用 带有条件码的指令可以很容易地实现分支程序。

例1：编写一个分支程序段，如果寄存器 R5 中的数据等于 10，就把 R5 中的数据存入寄存器 R1；否则把 R5 中的数据分别存入寄存器 R0 和 R1。

4.7 ARM汇编语言程序设计



4.7 ARM汇编语言程序设计

例1：编写一个分支程序段，如果寄存器R5中的数据等于10，就把R5中的数据存入寄存器 R1；否则把R5中的数据分别存入寄存器 R0和R1。

(1) 用 条件指令 实现的分支程序段

```
CMP  R5, #10
```

```
MOVNE R0, R5
```

```
MOV  R1, R5
```

(2) 用 条件转移指令 来实现分支程序段

```
CMP  R5, #10
```

```
BEQ  doequal
```

```
MOV  R0, R5
```

```
doequal  MOV  R1, R5
```

4.7 ARM汇编语言程序设计

例2：编写一个程序段，当寄存器R1中的数据**大于**R2中数据时，将R2中的数据加10存入寄存器 R1；否则将R2中数据加 5 存入寄存器 R1。

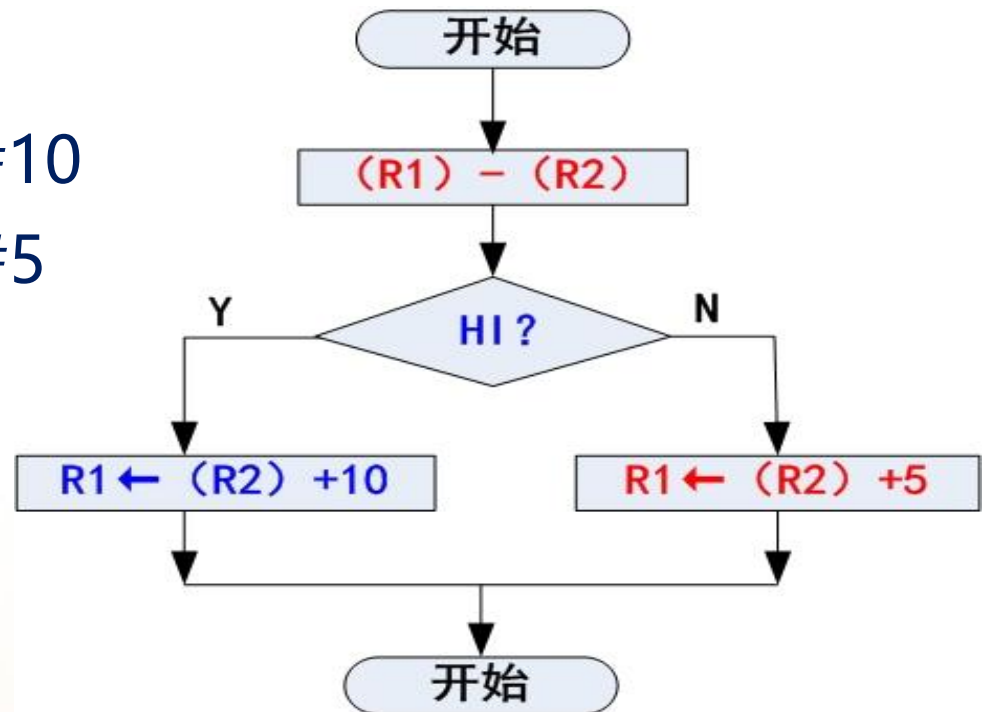
CMP R1, R2

ADDHI R1, R2, #10

ADDLS R1, R2, #5

(1) HI：无符号数大于，LS：无符号数小于或等于；

(2) GT：带符号数大于，LE：带符号数小于或等于。



4.7 ARM汇编语言程序设计

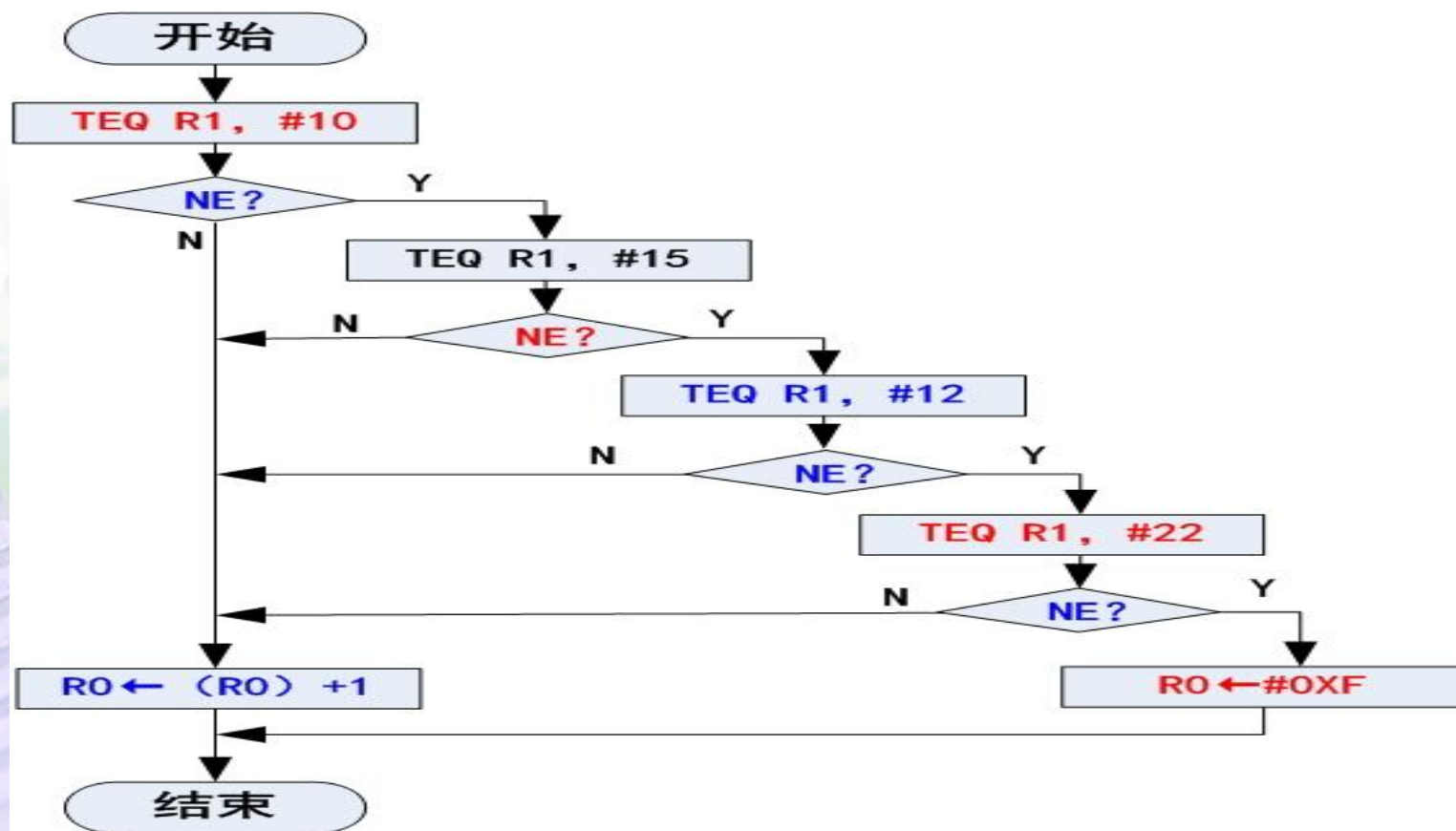
2、多分支（散转）程序设计

程序分支点上有多于两个以上的执行路径的程序叫做多分支程序。利用条件测试指令或跳转表可以实现多分支程序。

例1：编写一个程序段，判断寄存器R1中数据是否为10、15、12、22。如果是，则将R0中的数据加1；否则将R0设置为 0XF。

4.7 ARM汇编语言程序设计

例1：编写一个程序段，判断寄存器R1中数据是否为10、15、12、22。如果是，则将R0中的数据加1；否则将R0设置为0XF。



4.7 ARM汇编语言程序设计

```
MOV    R0, #0  
TEQ    R1, #10  
TEQNE  R1, #15  
TEQNE  R1, #12  
TEQNE  R1, #22  
ADDEQ  R0, R0, #1  
MOVNE  R0, #0XF
```

ADDEQ指令能带S吗？

4.7 ARM汇编语言程序设计

当多分支程序的每个分支所对应的是一个程序段时，常常把各个分支程序段的首地址依次存放在一个叫做**跳转地址表**的存储区域，然后在程序的**分支点**处使用一个可以将跳转表中的**目标地址**传送到PC的指令来实现分支。

一个具有 3 个分支的跳转地址表示意图如下：

基地址	TAB	0X00004540	FUN0
		0X00004544	FUN1
		0X00004548	FUN2
		0X0000454C	

4.7 ARM汇编语言程序设计

MOV R0, N ; N为表项序号0~2

ADR R5, JPTAB

LDR PC, [R5, R0, LSL #2]

JPTAB ; 跳转表

DCD FUN0

DCD FUN1

DCD FUN2

FUN0 ... ; 分支FUN0的程序段

FUN1 ... ; 分支FUN1的程序段

FUN2 ... ; 分支FUN2的程序段

4.7 ARM汇编语言程序设计

3、带ARM/Thumb状态切换的分支程序设计

在ARM程序中经常需要在程序跳转的同时还要进行处理器状态的转移，即从ARM指令程序段跳转到Thumb指令程序段（或相反）。为了实现这个功能，系统提供了一条专用的、可以实现4GB空间范围内的绝对跳转交换指令BX。

4.7 ARM汇编语言程序设计

下面是一段从 ARM 指令程序段跳转到 Thumb 指令程序的状态切换例程。

```
; ARM指令程序
```

```
CODE32
```

```
.....
```

```
ADR R0,Into_Thumb + 1
```

```
BX R0
```

```
.....
```

```
;Thumb指令程序
```

```
CODE16
```

```
Into_Thumb .....
```

4.7 ARM汇编语言程序设计

下面是一段从Thumb 指令程序段跳转到ARM指令程序的状态切换例程。

; Thumb 指令程序

CODE16

● ● ● ● ●

ADR R0,Back to ARM

BX R0

• • • • •

; ARM指令程序

CODE32

Back to ARM

4.7 ARM汇编语言程序设计

4.7.3 循环程序设计

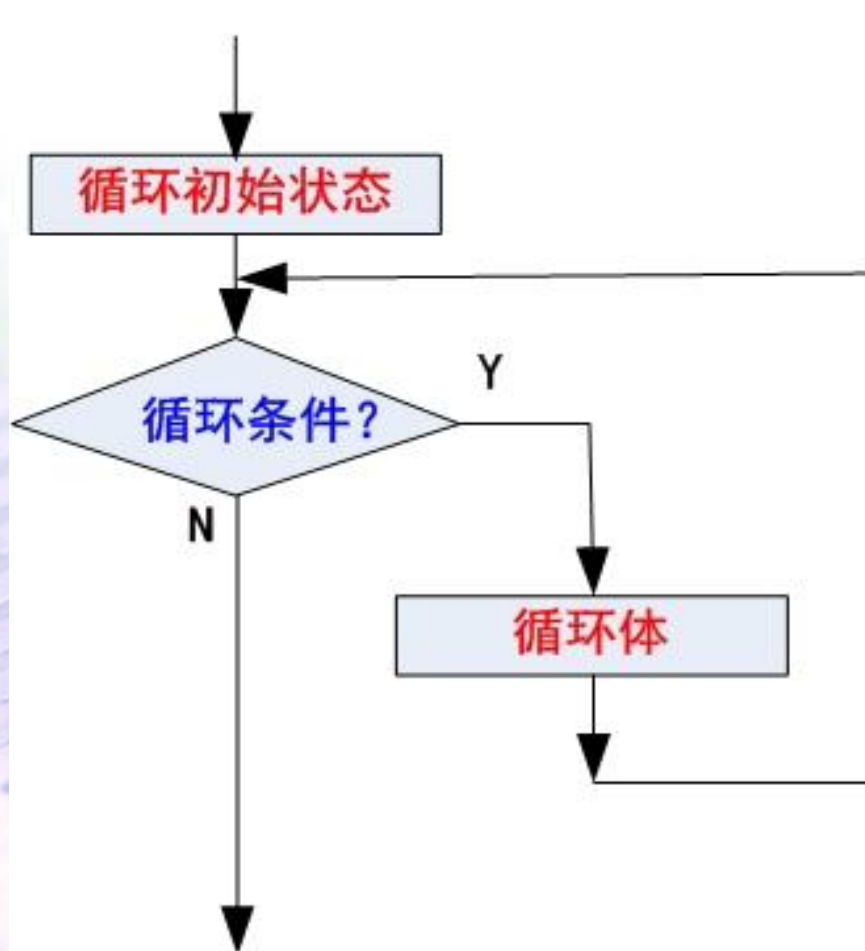
当条件满足时，需要重复执行同一个程序段做同样工作的程序叫做**循环程序**。

被重复执行的程序段叫做**循环体**，需要满足的条件叫做**循环条件**。

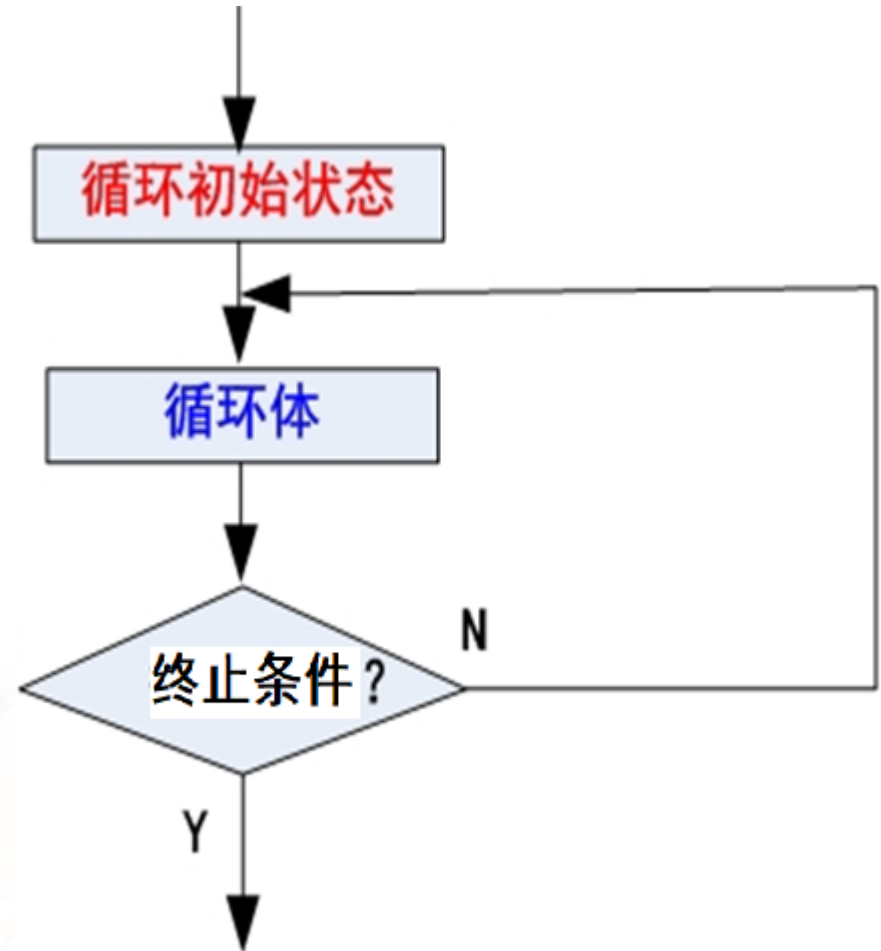
循环程序有两种结构：DO-WHILE结构 和 DO-UNTIL 结构。

4.7 ARM汇编语言程序设计

DO-WHILE结构



DO-UNTIL结构



4.7 ARM汇编语言程序设计

在汇编语言程序设计中，常用的是DO-UNTIL 结构循环程序。

```
MOV R1, #10  
LOOP      .....  
SUBS R1, R1, #1  
BNE  LOOP
```

4.7 ARM汇编语言程序设计

例1：编写一个程序，把首地址为DATA_SRC的80个字的数据复制到首地址为DATA_DST的目标数据块中。

```
LDR R1, =DATA_SRC
```

```
LDR R0, =DATA_DST
```

```
MOV R10, #10
```

```
LOOP
```

```
LDMIA R1! , {R2-R9}
```

```
STMIA R0! , {R2-R9}
```

```
SUBS R10, R10, #1
```

```
BNE LOOP
```

4.7 ARM汇编语言程序设计

4.7.4 子程序及其调用

1、子程序的调用与返回

人们把可以多次反复调用的、能完成指定功能的程序段称为“子程序”。把调用子程序的程序称为“主程序”。

为进行识别，子程序的第1条指令之前必须赋予一个标号，以便其他程序可以用这个标号调用子程序。

在ARM汇编语言程序中，主程序一般通过BL指令来调用子程序。

为使子程序执行完毕能返回主程序的调用处，子程序末尾处应有MOV、B、BX、LDMFD等指令，并在指令中将返回地址重新复制到PC中。

4.7 ARM汇编语言程序设计

例1：一个使用 MOV 指令实现返回的子程序。

relay

MOV PC, LR

使用 B 指令实现返回的子程序。

relay

B LR (错误)

可以使用BX LR (正确)

开发环境 不认这种。

4.7 ARM汇编语言程序设计

2、子程序中堆栈的使用

relay

STMFD R13!,{R0~R12,LR}; 压入堆栈

..... ; 子程序代码

LDMFD R13!,{R0~R12,PC} ; 弹出堆栈并返回

4.8 C/C++语言和汇编语言的混合编程

4.8.1 汇编程序访问全局C 变量

一般来说，汇编语言程序与C语言程序不在同一个文件上，所以实质上这是一个引用不同文件定义的变量问题。解决这个问题的办法就是使用关键字IMPORT和EXPORT。

在C程序中声明的全局变量可以被汇编程序通过地址间接访问，具体访问方法如下。

- ① 使用 IMPORT 伪指令声明该全局变量；
- ② 使用LDR宏指令读取该全局变量的内存地址，通常该全局变量的内存地址值存放在程序的数据缓冲池中；
- ③ 根据该数据的类型，使用相应的LDR指令读取该全局变量的值；使用相应的STR 指令修改该全局变量的值。

4.8 C/C++语言和汇编语言的混合编程

各数据类型及其对应的 LDR / STR 指令如下。

- ① 对于无符号的char类型的变量通过指令LDRB/STRB来读写；
- ② 对于无符号的short类型的变量通过指令LDRH / STRH 来读写；
- ③ 对于 int 类型的变量通过指令LDR / STR 来读写；
- ④ 对于有符号的char类型的变量通过指令LDRSB 来读取；
- ⑤ 对于有符号的char 类型的变量通过指令 STRB 来写入；
- ⑥ 对于有符号的short类型的变量通过指令 LDRSH 来读取；

4.8 C/C++语言和汇编语言的混合编程

- ⑦ 对于有符号的short类型的变量通过指令 STRH 来写入;
- ⑧ 对于小于8个字的结构型变量, 可以通过一条 LDM / STM 指令来读 / 写整个变量;
- ⑨ 对于结构型变量的数据成员, 可以使用相应的 LDR / STR 指令来访问, 这时必须知道该数据成员相对于结构型变量开始地址的偏移量。

4.8 C/C++语言和汇编语言的混合编程

例、下面是一个汇编代码的函数，它引用了一个在其他文件中定义的全局变量globvar，将其加 2 后写回 globvar。

```
AREA globals, CODE, READONLY
```

```
EXPORT asmsubroutine
```

```
IMPORT globvar
```

```
asmsubroutine
```

```
LDR R1,=globvar
```

```
LDR R0,[R1]
```

```
ADD R0,R0,#2
```

```
STR R0,[R1]
```

```
MOV PC,LR
```

```
END
```

4.8 C/C++语言和汇编语言的混合编程

4.8.2 C与汇编之间的函数调用

在ARM工程中，C程序调用汇编函数和汇编程序调用C函数是经常发生的事情。为此人们制定了ARM-Thumb过程调用标准 ATPCS （ ARM-Thumb Procedure Call Standard）。

4.8.2.1 ATPCS简介

为了使单独编译的C语言程序和汇编语言程序之间能够相互调用，必须为子程序间的调用设置一定的规则。具体包括：

- (1)在子程序编写时必须遵守相应的ATPCS规则。
- (2)数据栈的使用要遵守相应的ATPCS规则。
- (3)在汇编编译器中使用选项。

4.8 C/C++语言和汇编语言的混合编程

一、堆栈与寄存器在函数调用中的作用

函数是通过寄存器和堆栈来传递参数和返回值的。

下面是C语言程序调用C函数的情况。

```
int AddInt(int x, int y)
{
    int s;
    s = x + y;
    return s;
}
```

在C程序中，主函数main()调用该函数的方法如下：

```
void main(void)
{
    int a,b,x;
    x = AddInt(a,b); //调用
    .....
}
```

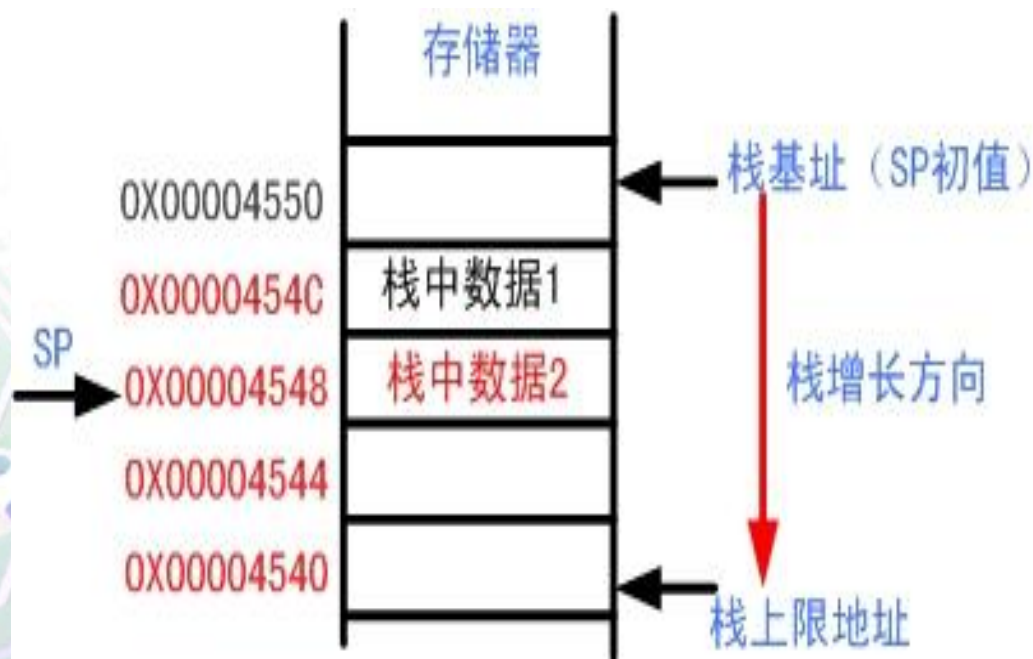
ARM编译器使用的函数调用规则就是ATPCS标准。ATPCS标准既是ARM编译器的规则，也是设计可被C程序调用的汇编函数的编写规则。

4.8 C/C++语言和汇编语言的混合编程

二、ATPCS 关于堆栈和寄存器的使用规则

ATPCS规定，ARM的数据堆栈为FD型堆栈，即递减满堆栈。

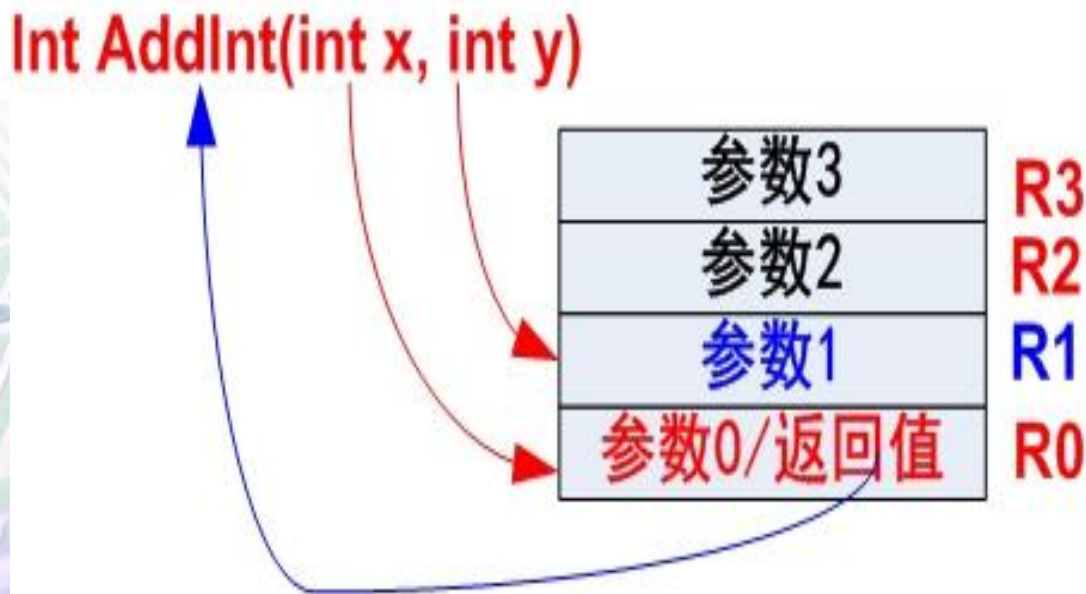
递减满堆栈示意图



4.8 C/C++语言和汇编语言的混合编程

ATPCS 标准规定，对于参数个数不多于4的函数，编译器必须按参数在列表中的顺序，**自左向右**为它们分配寄存器 **R0~R3**。其中函数返回时，**R0**还被用来存放函数的**返回值**。

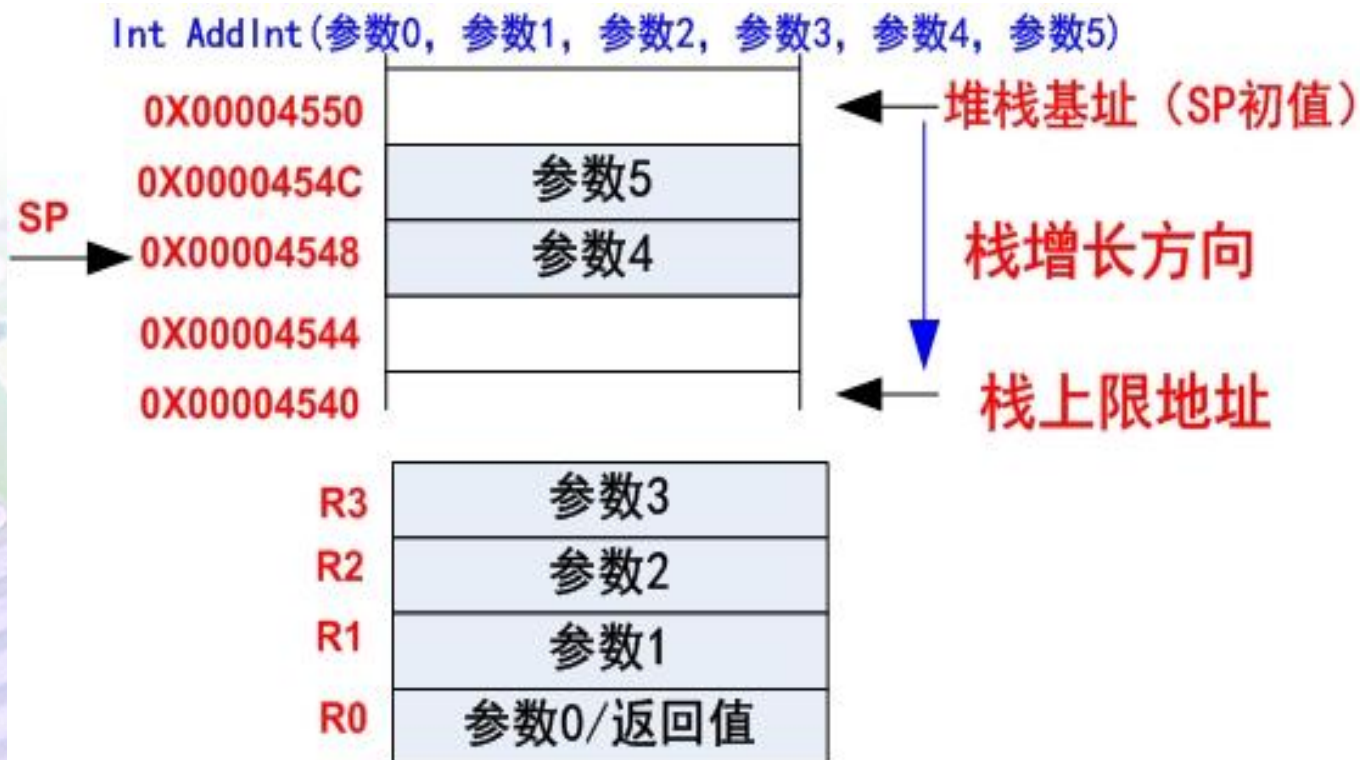
编译器为函数AddInt()的参数分配寄存器的示意图如下：



4.8 C/C++语言和汇编语言的混合编程

如果函数的参数多于4个，那么多余的参数则按自右向左的顺序压入数据堆栈，即参数入栈顺序与参数顺序相反。

函数参数使用寄存器和数据栈的示意图如下：



4.8 C/C++语言和汇编语言的混合编程

下表列举了ARM-Thumb过程调用标准规定的寄存器的名称和使用方法。

寄存器	别名1	别名2	用法
R0~R3	A1~A4		参数寄存器，其中R0又被用作函数返回值寄存器
R4~R8	V1~V5		函数局部变量寄存器
R9	V6	Sb	在RWPI情况下保存静态基地址
R10	V7	SI	用来保存堆栈边界地址
R11	V8	fp	保存结构指针
R12		lp	过度寄存器
R13		Sp	堆栈指针
R14		LR	连接寄存器
R15		PC	程序计数器

RWPI 读写位置无关（编译器选项）。

4.8 C/C++语言和汇编语言的混合编程

4.8.1.2 C程序可调用汇编函数实例

下面是一个用汇编语言编写的函数，该函数把R1指向的字符串复制到R0指向的存储块。

```
AREA tt, CODE, READONLY
```

```
EXPORT strcpy
```

```
strcpy
```

```
LDRB R2,[R1],#1
```

```
STRB R2,[R0],#1
```

```
CMP R2,#0
```

```
BNE strcpy
```

```
MOV PC,LR
```

```
END
```


4.8 C/C++语言和汇编语言的混合编程

根据ATPCS的C语言程序调用汇编函数的规则，参数由左向右依次传递给寄存器R0~R3，可知汇编函数strcpy在C程序中原型应该为：

```
void strcpy(char *d,const char* s);
```


4.8 C/C++语言和汇编语言的混合编程

在 C 语言文件中，调用 strcpy 函数的方法如下：

```
extern void strcpy(char *d,const char * s);  
  
int main(void)  
{  
    const char *src = "source" ;  
    char dest[10];  
    .....  
    strcpy(dest, src);  
    .....  
}
```

4.8 C/C++语言和汇编语言的混合编程

4.8.1.3 汇编程序调用C函数实例

现有 C 函数 g() 如下：

```
int g(int a, int b, int c, int d, int e){  
    return a+b+c+d+e;  
}
```

汇编函数f中调用C函数g(), 以实现下面的功能：

```
int f(int i) {return -g(i, 2*i, 3*i, 4*i,5*i)}
```

整个汇编函数 f 的代码如下：

```
EXPORT    f  
AREA     tt, CODE, READONLY  
IMPORT   g ; 声名g为外部引用符号  
ENTRY
```

4.8 C/C++语言和汇编语言的混合编程

f STR LR, [SP,#-4]!; 断点存入堆栈, 解决嵌套、返回
 ADD R1, R0, R0 ; (R1) = i*2
 ADD R2, R1, R0 ; (R2) = i*3
 ADD R3, R1, R2 ; (R3) = i*5
 STR R3, [SP, #-4]! ; 将 (R3) 即第5个参数i*5存入堆栈
 ADD R3, R1, R1 ; (R3) = i*4
 BL g ; 调用C函数g (), 返回值在寄存器R0中
 ADD SP, SP, #4 ; 清栈, 第5个参数
 RSB R0, R0, #0 ; 函数f的返回值 (R0) = 0 - (R0)
 LDR PC, [SP],#4 ; 恢复断点并返回
 END

4.8 C/C++语言和汇编语言的混合编程

4.8.2 C/C++语言和汇编语言的混合编程

除了上面介绍的函数调用方法之外，ARM编译器 armcc 中含有的内嵌汇编器还允许在C程序中内联或嵌入式汇编代码，以提高程序的效率。

4.8 C/C++语言和汇编语言的混合编程

4.8.2.1 内联汇编

1、定义内联汇编程序

所谓内联汇编程序，就是在C程序中**直接编写汇编程序段**而形成一个语句块，这个语句块可以使用除了 BX 和 BLX 之外的全部ARM指令来编写，从而可以使程序实现一些不能从C获得的底层功能。

其格式为：

```
--asm
{
    汇编语句块
}
```

4.8 C/C++语言和汇编语言的混合编程

例:

```
void enable_IRQ(void)
{
    int tmp;
    __asm    //声名内联汇编代码
    {
        MRS tmp, CPSR
        BIC tmp, tmp, #0x80
        MSR CPSR_c, tmp
    }
}
```


4.8 C/C++语言和汇编语言的混合编程

例：

```
void Read_mode(void)
{
    int tmp, RR;
    RR = 0xFFFFFEE0;
    __asm    //声名内联汇编代码
    {
        MRS tmp, CPSR
        BIC tmp, tmp, RR
    }
}
```

内联式汇编伪指令不能使用！

4.8 C/C++语言和汇编语言的混合编程

汇编语句块中，如果有两条指令占据了同一行，那么必须用分号 “ ; ” 将它们分隔。如果一条指令需要占用多行，那么必须用反斜线符号 “ \ ” 作为续行符。

可以在内联汇编语言块内的任意位置使用 C/C++ 格式的注释。

内联汇编代码中定义的标号可被用作跳转或 C/C++ goto 语句的目标，同样，在 C/C++ 代码中定义的标号，也可被用作内联汇编代码跳转指令的目标。

4.8 C/C++语言和汇编语言的混合编程

2、内联汇编的限制

内联汇编与真实汇编之间有很大区别，会受到很多限制。

- ① 它不支持 Thumb 指令；除了程序状态寄存器 CPSR 之外，不能直接访问其他任何物理寄存器等；
- ② 如果在内联汇编程序指令中出现了以某个寄存器名称命名的操作数，那么它被叫做虚拟寄存器，而不是实际的物理寄存器。编译器在生成和优化代码的过程中，会给每个虚拟寄存器分配实际的物理寄存器，但这个物理寄存器可能与在指令中指定的不同。唯一的一个例外就是状态寄存器PSR，任何对PSR的引用总是执行指向物理PSR；

4.8 C/C++语言和汇编语言的混合编程

例:

```
void enable_IRQ(void)
{
    //int tmp;
    __asm    //声名内联汇编代码
    {
        MRS R1, CPSR
        BIC R1, R1, #0x80
        MSR CPSR_c, R1
    }
}
```

4.8 C/C++语言和汇编语言的混合编程

- ③ 在内联汇编代码中不能使用寄存器 PC (R15)、LR (R14) 和SP (R13)，任何试图使用这些寄存器的操作都会导致出现错误消息；
- ④ 鉴于上述情况，在内联汇编语句块中最好使用C或C++变量作为操作数；
- ⑤ 虽然内联汇编代码可以更改处理器模式，但更改处理器模式会禁止使用C操作数或对已编译C代码的调用，直到将处理器模式恢复为原设置之后。

4.8 C/C++语言和汇编语言的混合编程

4.8.2.2 嵌入式汇编

嵌入式汇编程序是一个编写在C程序外的单独汇编程序，该程序段可以像函数那样被C程序调用。

与内联汇编不同，嵌入式汇编具有真实汇编的所有特性，数据交换符合 ATPCS 标准，同时支持 ARM 和Thumb，所以它可以对目标处理器进行不受限制的低级访问。但是不能直接引用 C/C++ 的变量。

用 `__asm` 声明的嵌入式汇编程序像C函数那样可以有参数和返回值。定义一个嵌入式汇编函数的语法格式为：

4.8 C/C++语言和汇编语言的混合编程

```
__asm return-type function-name(parameter-list)
{
    汇编程序段
}
```

return-type: 函数返回值类型, C语言中的数据类型;

function-name: 函数名;

parameter-list: 函数参数列表。

注: ADS环境中不能使用嵌入式汇编。

4.8 C/C++语言和汇编语言的混合编程

嵌入式汇编在形式上看起来就像使用关键字 `_ _asm` 进行了声明的函数，如下所示：

```
_ _asm int add(int i, int j)
{
    ADD R0, R0, R1
    MOV PC, LR
}
```

4.8 C/C++语言和汇编语言的混合编程

参数名只允许使用在参数列表中，不能用在嵌入式汇编函数体内。

如下面定义的嵌入式汇编程序是错误的。

```
__asm int f(int i)
{
    ADD i, i, #1 //错误
    MOV PC, LR
}
```

按 ATPCS 规定，应该使用寄存器 R0 来代替 i。

4.8 C/C++语言和汇编语言的混合编程

在C程序中调用嵌入式汇编程序的方法与调用C函数的方法相同。

```
void main()
{
    printf( "12345 + 67890 =%d\n" ,add(12345,67890));
}
```

灵活地使用内联汇编和嵌入式汇编，有助于提高程序效率。

4.8 C/C++语言和汇编语言的混合编程

4.8.2.3 内联汇编代码与嵌入式汇编代码之间的差异

- ① 内联汇编代码使用高级处理器抽象，并在代码生成过程中与 C 和 C++ 代码集成。因此编译程序将 C 和 C++ 代码与汇编代码一起进行优化；
- ② 与内联汇编代码不同，嵌入式汇编代码从 C 和 C++ 代码中分离出来单独进行汇编，产生与 C 和 C++ 源代码编译对象相结合的编译对象；
- ③ 可通过编译程序来内联汇编代码，但无论是显示还是隐式，都无法内联嵌入式汇编代码。

4.8 C/C++语言和汇编语言的混合编程

内联汇编程序与嵌入式汇编程序的主要差异

项目	内联汇编	嵌入式汇编
指令集	仅限于ARM	ARM和Thumb
ARM汇编程序命令 (伪指令)	不支持	支持
C表达式	支持	仅支持常量表达式
优化代码	支持	不支持
内联	可能	从不
寄存器访问	不使用物理寄存器	使用物理寄存器
返回指令	自动生成	显示编写

4.9 ARM系列处理器开发平台和工具(了解)

4.9.1 ARM公司提供的开发工具

4.9.1.1 ARM工具的发展历程

- ARM 公司成立早期，主要的开发工具是SDT (Software Development Toolkit) 。
- 1999年，SDT发展到V2.5版本之后，推出了全新的ADS 1.0 (ARM Developer Suite) 。
- 2002 年，ADS 发展到 1.2 之后，又推出了 RVDS (RealView Development Suite) 。RealView是一个工具的统称，其下包括RealView编译器 (RVCT)、调试器 (RVD)、仿真器等众多工具链。

4.9 ARM系列处理器开发平台和工具(了解)

2005年，ARM 收购 Keil 之后，ARM 工具就分为两派：
MDK 和 DS-5

- Keil 公司针对 ARM 的微控制器：RealView MDK (Microcontroller Development Kit)，后面叫 Keil MDK (或MDK-ARM)。
- ARM 公司升级 RVDS 为 DS-5 (ARM Development Studio 5)，支持所有ARM 内核处理器开发。目前的最新版本为ARM Development Studio 2022。

4.9 ARM系列处理器开发平台和工具(了解)

4.9.1.2 Microcontroller Development Kit (MDK)

原名RealView MDK，也称MDK-ARM、KEIL MDK、KEIL For ARM，都是同一个东西。ARM公司现在统一使用MDK-ARM的称呼，MDK的设备数据库中有很多厂商的芯片，是专为微控制器开发的工具，为满足基于MCU进行嵌入式软件开发的工程师需求而设计。

MDK ARM提供免费、基础、增强、专业四个版本，**只支持 Windows 操作系统。**

4.9 ARM系列处理器开发平台和工具(了解)

4.9.1.2 Microcontroller Development Kit (MDK)

最近，Keil 官方推出了一则消息：Keil MDK 新增一个版本，MDK社区版（MDK-Community edition）。

该版本主要有两个特点：

- 免费

- 没有代码大小限制

可供电子爱好者、学生、学者等群体非商业免费评估和使用。该版本具有Keil MDK Professional（专业版）的内容：单片机型号齐全，没有代码大小限制，其中的高级功能可以使用等。

<https://www.keil.arm.com/mdk-community/>

4.9 ARM系列处理器开发平台和工具(了解)

4.9.1.3 ARM Development Studio

ARM Development Studio是专为Arm架构打造的一款非常全面的端到端的嵌入式C/C++专用软件开发工具，能够与Arm处理器IP一起设计，从而加速Cortex-M、Cortex-R和Cortex-A处理器的系统设计和软件开发，能够帮助用户构建除更加强大而高效的产品。

下载链接：

<https://developer.arm.com/downloads/-/arm-development-studio-downloads>

4.9 ARM系列处理器开发平台和工具(了解)

4.9.1.4 MDK和ARM Development Studio比较

- ① IDE环境：MDK 基于 Keil μ Vision 开发环境。DS-5 基于 Eclipse 开发环境。
- ② 支持平台：MDK 仅支持 Windows 平台。DS-5 支持 Windows 和 Linux 平台。
- ③ 支持处理器：MDK 主要针对 ARM 的微控制器（MCU）；DS-5 针对 ARM 全系列处理器。

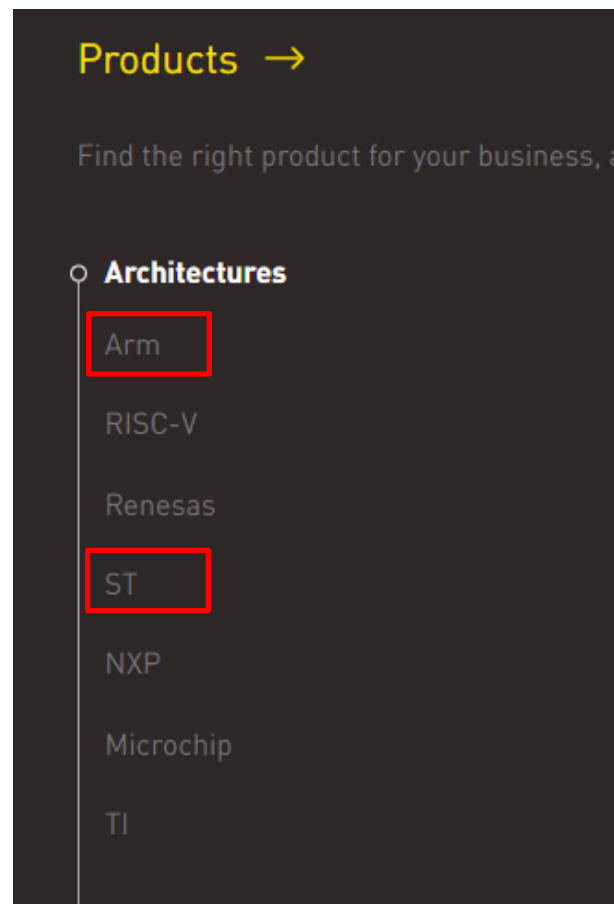
4.9 ARM系列处理器开发平台和工具(了解)

4.9.2 第三方开发工具

4.9.2.1 IAR

IAR是一家公司的名称，也是一种集成开发环境（IDE）的名称，我们平时所说的IAR主要是指的是集成开发环境，当然，我们也称它为一种工具：IAR开发工具。

IAR针对不同内核处理器，有不同的集成开发环境。



4.9 ARM系列处理器开发平台和工具(了解)

4.9.2.1 IAR

Development solutions for Arm

■ Development toolchain

IAR Embedded Workbench for Arm

User-friendly IDE
7,000+ supported Arm devices
Leading compiler technology
Comprehensive debugger

Free trial →

■ Build tools

IAR Build Tools for Arm

Cross-platform support
Flexible and high performance
7,000+ supported Arm devices
Modern workflows with cross-platform benefit

→

■ Certified toolchain

IAR Embedded Workbench for Arm, Functional Safety

Certified by TÜV SÜD
Simplified validation
Support for entire product lifecycle
Integrated code analysis add-ons

→

Supported architectures

Our development tools support these ST MCUs:

Arm

STM8

8051

Functional safety products for STM32 and STM8 MCUs

■ Certified toolchain

IAR Embedded Workbench for Arm, Functional Safety

Certified by TÜV SÜD
Simplified validation
Support for entire product lifecycle
Integrated code analysis add-ons

→

■ Certified toolchain

IAR Embedded Workbench for STM8, Functional Safety

Certified by TÜV SÜD
Simplified validation
Support for entire product lifecycle
Integrated static analysis add-on

→

■ Certified build tools

IAR Build Tools for Arm, Functional safety

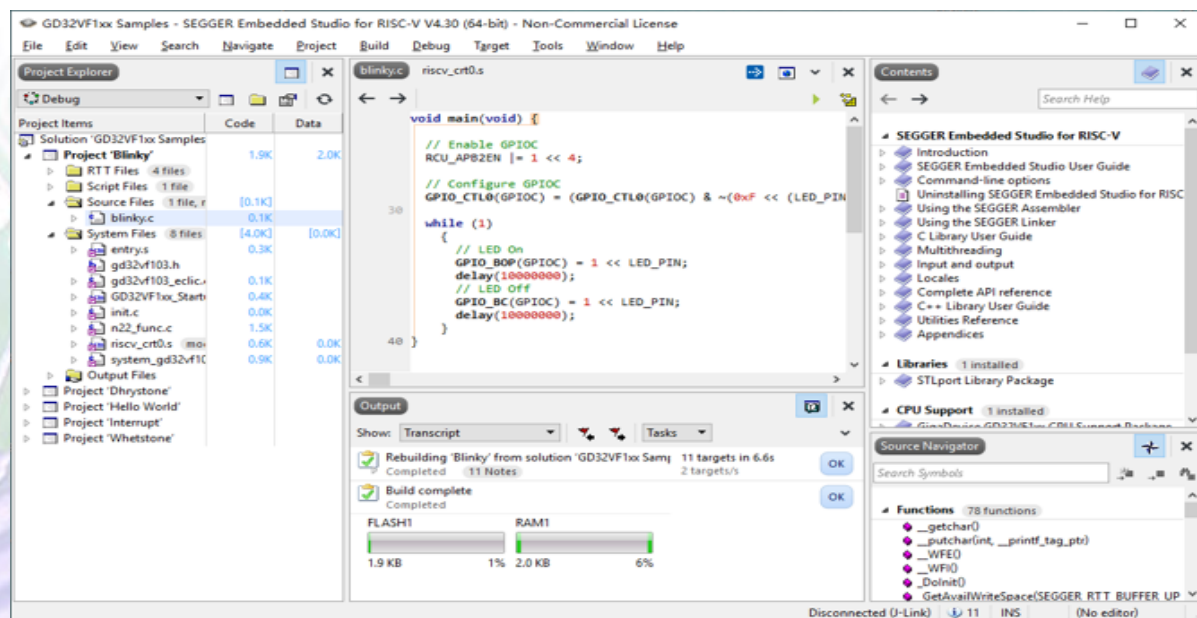
Certified by TÜV SÜD
Flexible and high performance
Support for entire product lifecycle
Modern workflows with cross-platform benefit

→

4.9 ARM系列处理器开发平台和工具(了解)

4.9.2.2 Embedded Studio集成开发环境

SEGGER Embedded Studio 是一款功能强大的 C/C++ 集成开发环境，支持ARM和RISC-V微控制器。专为嵌入式开发设计，提供一站式解决方案。



4.9 ARM系列处理器开发平台和工具(了解)

4.9.2.3 STM32集成开发工具STM32CubeIDE

2019年，ST推出了STM32CubeIDE集成开发环境。STM32CubeIDE是一个先进的C/C++开发平台，具有STM32微控制器的IP配置，代码生成，代码编译和调试功能。

它基于Eclipse/ CDT框架和用于开发的GCC工具链，以及用于调试的GDB。它允许集成数百个现有插件，完成EclipseIDE的功能。

<https://www.stmicroelectronics.com.cn/en/development-tools/stm32cubeide.html>

4.9 ARM系列处理器开发平台和工具(了解)

4.9.2.4 STM32的开发的四种方式

STM32的开发有四种方式：

- 1.寄存器版本
- 2.库函数版本
- 3.HAL库版本
- 4.LL库版本

4.9 ARM系列处理器开发平台和工具(了解)

4.9.2.4 STM32的开发的四种方式

1.寄存器版本

直接针对CPU的寄存器进行操作。

优点：效率最高

缺点：开发效率最低，最复杂

4.9 ARM系列处理器开发平台和工具(了解)

4.9.2.4 STM32的开发的四种方式

2.库函数版本

ST(意法半导体)推出了官方固件库，固件库将这些寄存器底层操作都封装起来，提供一整套接口供开发者调用。标准外设库（Standard Peripherals Library）是对STM32芯片的一个完整的封装，包括所有标准器件外设的器件驱动器。这应该是目前使用最多的ST库，几乎全部使用C语言实现。但是，标准外设库也是针对某一系列芯片而言的，**没有可移植性**。

优点：操作相对方便

缺点：执行效率中等

4.9 ARM系列处理器开发平台和工具(了解)

4.9.2.4 STM32的开发的四种方式

3.HAL库版本

HAL(Hardware Abstraction Layer)是硬件的抽象层,它表现出更高的抽象整合水平, HAL API集中关注各外设的公共函数功能, 这样便于定义一套通用的对用户友好的API函数接口, 从而可以轻松实现从一个STM32产品移植到另一个不同的STM32系列产品。

HAL库是ST未来主推的库, ST新出的芯片已经没有STD库了, 比如F7系列。现在, ST主推HAL库, 目前, HAL库已经支持STM32全线产品。总的来说, HAL库相对于库函数层次架构更加清晰, 更加抽象。

4.9 ARM系列处理器开发平台和工具(了解)

4.9.2.4 STM32的开发的四种方式

库函数和HAL库其实本质上都是将STM32的底层的寄存器进行封装并向用户提供友好的接口，这都极大的降低了用户的开发门槛。

但是相对于库函数来说，HAL库更加“通用”，能够较好的移植到其他的芯片上去，但也正是这样，导致其代码比较庞大、执行效率比较低的结果。总的来说，HAL库相对于库函数更加友好，能够让用户将精力放在开发的产品上而不是怎么实现。

ST专门为其开发了配套的桌面软件**STMCubeMX**，开发者可以直接使用该软件进行可视化配置，大大节省开发时间。

4.9 ARM系列处理器开发平台和工具(了解)

4.9.2.4 STM32的开发的四种方式

4.LL库版本

LL库 (Low Layer) 是ST最近新增的库，与HAL库捆绑发布。LL 库更接近硬件层，对需要复杂上层协议栈的外设不适用，直接操作寄存器。其实现方法更加高效、简洁。使用LL库编程和使用标准外设库的方式基本一样，但是会得到比标准外设库更高的效率和更紧凑的代码。

独立使用，该库完全独立实现，可以完全抛开HAL库，只用LL库编程完成。在使用STM32CubeMX生成项目时，直接选LL库即可。如果使用了复杂的外设，例如 USB，则会调用 HAL库混合使用，和HAL 库结合使用。

作业

作业见群文件《平时作业要求》中第4章的内容。